

LAGERSYSTEM

Code-Dokumentation

Struktur, Module und vollstaendiger Quellcode
der Python/Streamlit-Anwendung

Analog aufgebaut wie die urspruengliche Dokumentation des Excel/VBA-Systems: je Datei/Modul zunaechst eine kurze Funktions-Uebersicht, danach der vollstaendige, unveraenderte Quellcode.

Repository: github.com/houcinehassine/lagersystem (privat)

Live-App: lagersystem.streamlit.app

Stand: 16. August 2026

TEIL 1

Projektstruktur

Ueberblick ueber alle Dateien des Projekts

Die Anwendung ersetzt die Excel-Arbeitsmappe durch eine schlanke Drei-Schichten-Architektur: die Streamlit-Oberflaeche (app.py) ruft ausschliesslich Funktionen aus der Geschäftslogik (logic.py) auf, diese wiederum liest/schreibt ueber die Datenzugriffsschicht (db.py) - nie direktes SQL in app.py oder logic.py an der Datenbank vorbei. So bleibt jede Schicht unabhaengig testbar und austauschbar (z.B. SQLite lokal, Postgres in der Cloud, ohne dass app.py oder logic.py etwas davon merken).

Funktion / Bereich	Zweck
app.py	Streamlit-Oberflaeche: Seiten, Formulare, Dialoge, Navigation. Ersetzt die Excel-Blaetter + UserForms.
logic.py	Geschäftslogik: Barcode-Vergabe, Materialentnahme, Suche, Export. Portierung der VBA-Module E/F/G/H/B/C.
db.py	Datenzugriffsschicht: verbindet wahlweise mit SQLite (lokal) oder Postgres (Cloud), reine SQL-Helfer (fetch_all, execute, ...).
schema.sql	Tabellendefinition fuer die lokale SQLite-Datenbank.
schema_postgres.sql	Inhaltlich identisches Tabellenschema fuer Postgres (Cloud/Neon).
migrate_sqlite_to_postgres.py	Einmaliges Migrations-Script: uebertraegt bestehende SQLite-Daten in eine Postgres-Datenbank.
backup.sh	Taeglicher Backup-Job (Mac/Linux): kopiert lager.db mit Zeitstempel weg.
server.sh	Startet den Streamlit-Server im lokalen Netzwerk (Mac/Linux).
start.command	Doppelklick-Startpunkt fuer macOS (ruft server.sh auf).
windows-setup/	Windows-Pendants: start.bat, start_server.bat, start_hidden.vbs, backup.py.

TEIL 2

Datenbankschema

schema.sql und schema_postgres.sql

Beide Schema-Dateien sind inhaltlich identisch - gleiche Tabellen, gleiche Spalten, gleiche Reihenfolge - und unterscheiden sich nur in Postgres-spezifischer Syntax (SERIAL statt AUTOINCREMENT, keine PRAGMA-Zeilen). db.py waehlt beim Start automatisch die passende Datei, abhaengig davon, ob eine DATABASE_URL konfiguriert ist.

schema.sql

Tabellenschema fuer die lokale SQLite-Datenbank (Normalbetrieb Mac/Windows).

```
-- =====
-- Lager System - SQLite Datenbankschema
-- Entspricht 1:1 den bestehenden Excel-Tabellen (ListObjects)
-- Wird von db.py beim ersten Start automatisch ausgefuehrt
-- =====

PRAGMA foreign_keys = ON;

-- -----
-- Lookup-Tabellen (bisher im Blatt "Einstellung")
-- -----

CREATE TABLE IF NOT EXISTS Lager (
    LagerID      INTEGER PRIMARY KEY AUTOINCREMENT,
    OrtRegal     TEXT NOT NULL UNIQUE
);

CREATE TABLE IF NOT EXISTS Typ (
    TypID        INTEGER PRIMARY KEY AUTOINCREMENT,
    Bezeichnung  TEXT NOT NULL UNIQUE           -- "Voll" / "Rest"
);

CREATE TABLE IF NOT EXISTS Einheit (
    EinheitID    INTEGER PRIMARY KEY AUTOINCREMENT,
    Symbol       TEXT NOT NULL UNIQUE           -- z.B. Stk, kg, m
);

CREATE TABLE IF NOT EXISTS Konfiguration (
    Schluessel   TEXT PRIMARY KEY,             -- z.B. "SchrottGrenzeMM"
    wert         TEXT NOT NULL                  -- als Text gespeichert, je nach Schluessel interpretiert
);

CREATE TABLE IF NOT EXISTS Profil_Abk (
    Profil       TEXT PRIMARY KEY,             -- voller Profilname
    Abkuerzung   TEXT NOT NULL                  -- Kuerzel fuer Barcode
);

CREATE TABLE IF NOT EXISTS Material_Gruppe (
    Material     TEXT PRIMARY KEY,
    Gruppe      TEXT NOT NULL,
    Abkuerzung   TEXT NOT NULL                  -- Kuerzel fuer Barcode
);

-- -----
-- Artikel_Gruppe (bisher Blatt "Artikel Gruppe")
-- Stammdaten je Artikeltyp, generiert den Haupt-Barcode
-- -----

CREATE TABLE IF NOT EXISTS Artikel_Gruppe (
    Barcode      TEXT PRIMARY KEY,             -- Haupt-Barcode, z.B. "LPAL0001"
    Bezeichnung  TEXT NOT NULL,                -- Profil-Mass-Material
    Profil       TEXT NOT NULL,
    Mass        TEXT NOT NULL,
    Material     TEXT NOT NULL
);

-- -----
-- Artikel_Liste (volle Stuecke), Reste (< 1000mm), Verlauf (Protokoll)
```

-- Alle drei identisch aufgebaut (ArtikelStueck-Typ aus dem VBA-Code)

```
-----
CREATE TABLE IF NOT EXISTS Artikel_Liste (
  ID                INTEGER PRIMARY KEY AUTOINCREMENT,
  Datum             TEXT NOT NULL,                -- ISO-Format, SQLite kennt kein natives DATETIME
  Barcode           TEXT NOT NULL,
  Barcode2          TEXT NOT NULL UNIQUE,
  Bezeichnung       TEXT,
  Profil            TEXT,
  Mass              TEXT,
  LaengeMM          REAL,
  MaterialGruppe    TEXT,
  Material          TEXT,
  OrtRegal          TEXT,
  Typ               TEXT,                        -- "Voll" / "Rest"
  MengeStueck       INTEGER,
  EPreis            REAL,
  EPreisEinheit     TEXT,
  FOREIGN KEY (Barcode) REFERENCES Artikel_Gruppe (Barcode)
);
```

```
CREATE TABLE IF NOT EXISTS Reste (
  ID                INTEGER PRIMARY KEY AUTOINCREMENT,
  Datum             TEXT NOT NULL,
  Barcode           TEXT NOT NULL,
  Barcode2          TEXT NOT NULL UNIQUE,
  Bezeichnung       TEXT,
  Profil            TEXT,
  Mass              TEXT,
  LaengeMM          REAL,
  MaterialGruppe    TEXT,
  Material          TEXT,
  OrtRegal          TEXT,
  Typ               TEXT,
  MengeStueck       INTEGER,
  EPreis            REAL,
  EPreisEinheit     TEXT,
  FOREIGN KEY (Barcode) REFERENCES Artikel_Gruppe (Barcode)
);
```

```
CREATE TABLE IF NOT EXISTS Papierkorb (
  ID                INTEGER PRIMARY KEY AUTOINCREMENT,
  QuellTabelle      TEXT NOT NULL,                -- "Artikel_Liste" oder "Reste" - fuer die Wiederherstellung
  GeloeschtAm       TEXT NOT NULL,
  Datum             TEXT,
  Barcode           TEXT,
  Barcode2          TEXT,
  Bezeichnung       TEXT,
  Profil            TEXT,
  Mass              TEXT,
  LaengeMM          REAL,
  MaterialGruppe    TEXT,
  Material          TEXT,
  OrtRegal          TEXT,
  Typ               TEXT,
  MengeStueck       INTEGER,
  EPreis            REAL,
  EPreisEinheit     TEXT
);
```

```
CREATE TABLE IF NOT EXISTS Verlauf (
  ID                INTEGER PRIMARY KEY AUTOINCREMENT,
  Datum             TEXT NOT NULL,
  Barcode           TEXT NOT NULL,
  Barcode2          TEXT NOT NULL,
  Bezeichnung       TEXT,
  Profil            TEXT,
  Mass              TEXT,
  LaengeMM          REAL,                        -- negativ = Entnahme (wie im bestehenden Code)
  MaterialGruppe    TEXT,
  Material          TEXT,
  OrtRegal          TEXT,
  Typ               TEXT,                        -- "Voll" / "Rest" / "Verbrauch"
  MengeStueck       INTEGER,
  EPreis            REAL,
  EPreisEinheit     TEXT
);
```

-- -----
-- Indizes fuer die Felder, die am haeufigsten durchsucht werden
-- (bisher per VBA-Schleife "Rowsuchen" durchsucht - jetzt per Index)
-- -----

```

CREATE INDEX IF NOT EXISTS idx_ArtikelListe_Barcode ON Artikel_Liste (Barcode);
CREATE INDEX IF NOT EXISTS idx_Reste_Barcode ON Reste (Barcode);
CREATE INDEX IF NOT EXISTS idx_Verlauf_Barcode ON Verlauf (Barcode);

```

schema_postgres.sql

Identisches Schema fuer Postgres - wird automatisch verwendet, sobald eine DATABASE_URL (z.B. Neon) konfiguriert ist.

```

-- =====
-- Lager System - PostgreSQL-Datenbankschema (fuer Cloud-Betrieb,
-- z.B. Neon). Inhaltlich identisch zu schema.sql (SQLite-Version),
-- nur mit Postgres-kompatibler Syntax (SERIAL statt AUTOINCREMENT,
-- keine PRAGMA-Zeilen). Wird von db.py automatisch verwendet, sobald
-- eine DATABASE_URL (Postgres-Verbindung) konfiguriert ist.
-- =====

-- -----
-- Lookup-Tabellen (bisher im Blatt "Einstellung")
-- -----

CREATE TABLE IF NOT EXISTS Lager (
    LagerID      SERIAL PRIMARY KEY,
    OrtRegal     TEXT NOT NULL UNIQUE
);

CREATE TABLE IF NOT EXISTS Typ (
    TypID        SERIAL PRIMARY KEY,
    Bezeichnung  TEXT NOT NULL UNIQUE           -- "Voll" / "Rest"
);

CREATE TABLE IF NOT EXISTS Einheit (
    EinheitID    SERIAL PRIMARY KEY,
    Symbol       TEXT NOT NULL UNIQUE           -- z.B. Stk, kg, m
);

CREATE TABLE IF NOT EXISTS Konfiguration (
    Schluessel   TEXT PRIMARY KEY,             -- z.B. "SchrottGrenzeMM"
    Wert         TEXT NOT NULL                  -- als Text gespeichert, je nach Schluessel interpretiert
);

CREATE TABLE IF NOT EXISTS Profil_Abk (
    Profil       TEXT PRIMARY KEY,             -- voller Profilname
    Abkuerzung   TEXT NOT NULL                  -- Kuerzel fuer Barcode
);

CREATE TABLE IF NOT EXISTS Material_Gruppe (
    Material     TEXT PRIMARY KEY,
    Gruppe       TEXT NOT NULL,
    Abkuerzung   TEXT NOT NULL                  -- Kuerzel fuer Barcode
);

-- -----
-- Artikel_Gruppe (bisher Blatt "Artikel Gruppe")
-- Stammdaten je Artikeltyp, generiert den Haupt-Barcode
-- -----

CREATE TABLE IF NOT EXISTS Artikel_Gruppe (
    Barcode      TEXT PRIMARY KEY,             -- Haupt-Barcode, z.B. "LPAL0001"
    Bezeichnung   TEXT NOT NULL,               -- Profil-Mass-Material
    Profil       TEXT NOT NULL,
    Mass         TEXT NOT NULL,
    Material     TEXT NOT NULL
);

-- -----
-- Artikel_Liste (volle Stuecke), Reste (< 1000mm), Verlauf (Protokoll)
-- Alle drei identisch aufgebaut (ArtikelStueck-Typ aus dem VBA-Code)
-- -----

CREATE TABLE IF NOT EXISTS Artikel_Liste (
    ID           SERIAL PRIMARY KEY,
    Datum       TEXT NOT NULL,
    Barcode     TEXT NOT NULL,
    Barcode2    TEXT NOT NULL UNIQUE,
    Bezeichnung  TEXT,
    Profil      TEXT,
    Mass        TEXT,
    LaengeMM    REAL,

```

```

MaterialGruppe TEXT,
Material       TEXT,
OrtRegal       TEXT,
Typ            TEXT,                -- "Voll" / "Rest"
MengeStueck    INTEGER,
EPreis        REAL,
EPreisEinheit  TEXT,
FOREIGN KEY (Barcode) REFERENCES Artikel_Gruppe (Barcode)
);

CREATE TABLE IF NOT EXISTS Reste (
ID            SERIAL PRIMARY KEY,
Datum        TEXT NOT NULL,
Barcode      TEXT NOT NULL,
Barcode2     TEXT NOT NULL UNIQUE,
Bezeichnung  TEXT,
Profil       TEXT,
Mass         TEXT,
LaengeMM     REAL,
MaterialGruppe TEXT,
Material     TEXT,
OrtRegal     TEXT,
Typ          TEXT,
MengeStueck  INTEGER,
EPreis       REAL,
EPreisEinheit TEXT,
FOREIGN KEY (Barcode) REFERENCES Artikel_Gruppe (Barcode)
);

CREATE TABLE IF NOT EXISTS Papierkorb (
ID            SERIAL PRIMARY KEY,
QuellTabelle TEXT NOT NULL,        -- "Artikel_Liste" oder "Reste" - fuer die Wiederherstellung
GeloeschtAm   TEXT NOT NULL,
Datum        TEXT,
Barcode      TEXT,
Barcode2     TEXT,
Bezeichnung  TEXT,
Profil       TEXT,
Mass         TEXT,
LaengeMM     REAL,
MaterialGruppe TEXT,
Material     TEXT,
OrtRegal     TEXT,
Typ          TEXT,
MengeStueck  INTEGER,
EPreis       REAL,
EPreisEinheit TEXT
);

CREATE TABLE IF NOT EXISTS Verlauf (
ID            SERIAL PRIMARY KEY,
Datum        TEXT NOT NULL,
Barcode      TEXT NOT NULL,
Barcode2     TEXT NOT NULL,
Bezeichnung  TEXT,
Profil       TEXT,
Mass         TEXT,
LaengeMM     REAL,                -- negativ = Entnahme (wie im bestehenden Code)
MaterialGruppe TEXT,
Material     TEXT,
OrtRegal     TEXT,
Typ          TEXT,                -- "Voll" / "Rest" / "Verbrauch"
MengeStueck  INTEGER,
EPreis       REAL,
EPreisEinheit TEXT
);

-- -----
-- Indizes fuer die Felder, die am haeufigsten durchsucht werden
-- -----

CREATE INDEX IF NOT EXISTS idx_ArtikelListe_Barcode ON Artikel_Liste (Barcode);
CREATE INDEX IF NOT EXISTS idx_Reste_Barcode      ON Reste (Barcode);
CREATE INDEX IF NOT EXISTS idx_Verlauf_Barcode     ON Verlauf (Barcode);

```

db.py

Kapselt den kompletten Datenbankzugriff. Waehlt automatisch zwischen lokalem SQLite und Cloud-Postgres, abhaengig von der Umgebungsvariable `DATABASE_URL` - der Rest der App merkt davon nichts.

Funktions-Uebersicht

Funktion / Bereich	Zweck
<code>_hole_database_url()</code>	Liest die Postgres-Verbindung aus Streamlit-Secrets oder Umgebungsvariable.
<code>_ist_postgres()</code>	
<code>_uebersetze()</code>	
<code>get_connection()</code>	Liefert eine neue DB-Verbindung - Postgres oder lokales SQLite, je nach <code>DATABASE_URL</code> .
<code>init_db()</code>	Legt beim Start alle Tabellen an (falls noetig) und fuehlt Standardwerte.
<code>_seed_defaults()</code>	Fuehlt Typ/Einheit mit Standardwerten, falls die Tabellen noch leer sind.
<code>connection()</code>	Context-Manager: oeffnet eine Verbindung, committed automatisch, schliesst sie danach.
<code>class _Zeile</code>	
<code>_normalisiere_zeile()</code>	Gleicht die Gross-/Kleinschreibung einer Postgres-Zeile ans Schema an.
<code>_normalisiere_df()</code>	Gleicht DataFrame-Spaltennamen (Postgres liefert sie klein) ans Schema an.
<code>_lade_schema_spalten()</code>	Extrahiert die echten Spaltennamen (Schreibweise) aus <code>schema_postgres.sql</code> .
<code>_cursor()</code>	Liefert einen Cursor mit dict-artigem Zeilenzugriff (<code>row['Spalte']</code>).
<code>fetch_all()</code>	Fuehrt eine SELECT-Abfrage aus, liefert alle Zeilen als Liste.
<code>fetch_one()</code>	Fuehrt eine SELECT-Abfrage aus, liefert die erste Zeile (oder None).
<code>fetch_df()</code>	Wie <code>fetch_all</code> , aber direkt als pandas-DataFrame - fuer die Streamlit-Tabellen.
<code>execute()</code>	Fuehrt INSERT/UPDATE/DELETE aus, liefert die Anzahl betroffener Zeilen.
<code>value_exists()</code>	Prueft, ob ein Wert in einer Spalte schon existiert (Duplikat-Check).

Funktion / Bereich	Zweck
hole_konfiguration()	Liest einen konfigurierbaren Wert (z.B. Schrott-Grenze), mit Standardwert.
setze_konfiguration()	Speichert/aktualisiert einen konfigurierbaren Wert.

Vollstaendiger Quellcode

Modulkopf / Moduldaten

```

"""Datenzugriffsschicht fuer das Lager System.

Unterstuetzt zwei Betriebsarten, automatisch anhand der Umgebungsvariable
DATABASE_URL gewaehlt:

- Keine DATABASE_URL gesetzt (Normalfall lokal auf Mac/Windows): lokale
  SQLite-Datei (lager.db) - wie bisher, keine Aenderung am Verhalten.
- DATABASE_URL gesetzt (z.B. via Streamlit-Secrets in der Cloud, verweist
  auf eine gehostete Postgres-Datenbank wie Neon): Postgres.

Der Rest der App (logic.py, app.py) merkt davon nichts - die Funktionen
hier (fetch_all, fetch_one, fetch_df, execute, ...) verhalten sich in
beiden Faellen gleich.
"""
from __future__ import annotations

import os
import sqlite3
from contextlib import contextmanager
from pathlib import Path

import pandas as pd
import psycopg2
import psycopg2.extras

BASE_DIR = Path(__file__).resolve().parent
DB_PATH = BASE_DIR / "lager.db"
SCHEMA_PATH_SQLITE = BASE_DIR / "schema.sql"
SCHEMA_PATH_POSTGRES = BASE_DIR / "schema_postgres.sql"

```

def _hole_database_url(...)

Liest die Postgres-Verbindung aus Streamlit-Secrets oder Umgebungsvariable.

```

def _hole_database_url() -> str | None:
    """Postgres-Verbindung fuer Cloud-Betrieb - zuerst aus Streamlit-Secrets
    (so wird sie auf Streamlit Community Cloud hinterlegt), sonst aus einer
    normalen Umgebungsvariable (praktisch zum lokalen Testen). Ist beides
    nicht gesetzt, bleibt es beim lokalen SQLite-Betrieb."""
    try:
        import streamlit as st

        if "DATABASE_URL" in st.secrets:
            return st.secrets["DATABASE_URL"]
    except Exception:
        pass
    return os.environ.get("DATABASE_URL")

```

Modulkopf / Moduldaten

```

DATABASE_URL = _hole_database_url()

# Sowohl psycopg2.IntegrityError als auch sqlite3.IntegrityError erfuehlen die
# DB-API-2.0-Schnittstelle - Aufrufer (app.py) fangen einheitlich db.IntegrityError,
# ohne wissen zu muessen, welcher Datenbanktyp gerade aktiv ist.
IntegrityError = psycopg2.IntegrityError if DATABASE_URL else sqlite3.IntegrityError

```

def _ist_postgres(...)

```

def _ist_postgres() -> bool:
    return DATABASE_URL is not None

```

def _uebersetze(...)

```
def _uebersetze(query: str) -> str:
    """Uebersetzt SQLite-Platzhalter (?) in Postgres-Platzhalter (%s), falls noetig."""
    return query.replace("?", "%s") if _ist_postgres() else query
```

def get_connection(...)

Liefert eine neue DB-Verbindung - Postgres oder lokales SQLite, je nach DATABASE_URL.

```
def get_connection():
    if _ist_postgres():
        # Kein cursor_factory hier auf Verbindungsebene - pandas' read_sql_query
        # (siehe fetch_df) braucht einen normalen Tupel-Cursor. Fuer unsere
        # eigenen dict-artigen Zugriffe (row["Spalte"]) siehe _cursor() unten.
        return psycopg2.connect(DATABASE_URL)

    conn = sqlite3.connect(DB_PATH, check_same_thread=False)
    conn.row_factory = sqlite3.Row
    conn.execute("PRAGMA foreign_keys = ON")
    # WAL erlaubt gleichzeitiges Lesen waehrend ein anderer Nutzer schreibt -
    # wichtig, sobald mehrere Leute im Netzwerk parallel auf die App zugreifen.
    conn.execute("PRAGMA journal_mode = WAL")
    # Bei kurzzeitig konkurrierenden Schreibzugriffen bis zu 5s warten statt
    # sofort mit "database is locked" abubrechen.
    conn.execute("PRAGMA busy_timeout = 5000")
    return conn
```

def init_db(...)

Legt beim Start alle Tabellen an (falls noetig) und fuellt Standardwerte.

```
def init_db() -> None:
    """Legt alle Tabellen an, falls noch nicht vorhanden, und fuellt Standardwerte."""
    conn = get_connection()
    try:
        schema_pfad = SCHEMA_PATH_POSTGRES if _ist_postgres() else SCHEMA_PATH_SQLITE
        skript = schema_pfad.read_text(encoding="utf-8")
        if _ist_postgres():
            conn.cursor().execute(skript)
        else:
            conn.executescript(skript)
        _seed_defaults(conn)
        conn.commit()
    finally:
        conn.close()
```

def _seed_defaults(...)

Fuellt Typ/Einheit mit Standardwerten, falls die Tabellen noch leer sind.

```
def _seed_defaults(conn) -> None:
    """Fuellt Typ/Einheit mit sinnvollen Standardwerten, falls noch leer.

    Lager/Profil_Abk/Material_Gruppe sind projektspezifisch und werden
    ueber die Einstellungen-Seite in der App gepflegt.
    """
    cur = _cursor(conn)

    cur.execute("SELECT COUNT(*) AS anzahl FROM Typ")
    if cur.fetchone()["anzahl"] == 0:
        cur.executemany(
            _uebersetze("INSERT INTO Typ (Bezeichnung) VALUES (?)", [("Voll",), ("Rest",)])
        )

    cur.execute("SELECT COUNT(*) AS anzahl FROM Einheit")
    if cur.fetchone()["anzahl"] == 0:
        cur.executemany(
            _uebersetze("INSERT INTO Einheit (Symbol) VALUES (?)",
                [("Stk",), ("kg",), ("m",), ("mm",)])
        )
```

def connection(...)

Context-Manager: oeffnet eine Verbindung, committed automatisch, schliesst sie danach.

```
@contextmanager
def connection():
    """Context-Manager fuer eine kurzlebige Verbindung (Commit/Close automatisch)."""
    conn = get_connection()
    try:
        yield conn
    finally:
        conn.commit()
```

```
finally:
    conn.close()
```

Modulkopf / Moduldaten

```
# -----
# Postgres faltet unquotierte Bezeichner (Tabellen/Spalten) automatisch auf
# Kleinschreibung - "LaengeMM" kommt als "laengemm" zurueck. sqlite3.Row
# erlaubt Zugriff per Position (row[0]) UND per Name in der im Schema
# definierten Gross-/Kleinschreibung. _Zeile bildet das fuer Postgres nach -
# und zwar, indem die Schluessel schon beim Erstellen umbenannt werden
# (nicht erst bei row["Spalte"]), damit auch dict(row) (z.B. in
# logic.hole_daten_ag) die richtige Schreibweise behaelt.
# -----
```

class _Zeile

```
class _Zeile(dict):
    def __init__(self, mapping):
        self._werte = list(mapping.values())
        super().__init__(
            (_SCHEMA_SPALTEN.get(k.lower(), k), v) for k, v in mapping.items()
        )

    def __getitem__(self, schluessel):
        if isinstance(schluessel, int):
            return self._werte[schluessel]
        return super().__getitem__(schluessel)
```

def _normalisiere_zeile(...)

Gleicht die Gross-/Kleinschreibung einer Postgres-Zeile ans Schema an.

```
def _normalisiere_zeile(row):
    if row is None or not _ist_postgres():
        return row
    return _Zeile(row)
```

def _normalisiere_df(...)

Gleicht DataFrame-Spaltennamen (Postgres liefert sie klein) ans Schema an.

```
def _normalisiere_df(df: pd.DataFrame) -> pd.DataFrame:
    """DataFrame-Spaltennamen fuer die Anzeige in die im Schema definierte
    Gross-/Kleinschreibung zurueckbringen (Postgres liefert sie klein)."""
    if _ist_postgres():
        df = df.rename(columns={c: _SCHEMA_SPALTEN.get(c.lower(), c) for c in df.columns})
    return df
```

Modulkopf / Moduldaten

```
# Fuer _normalisiere_df: nur echte Schema-Spalten umbenennen (nicht per
# Alias erzeugte Anzeigespalten wie "Quelle") - aus dem Postgres-Schema
# extrahiert, damit es garantiert mit den echten Tabellen uebereinstimmt.
```

def _lade_schema_spalten(...)

Extrahiert die echten Spaltennamen (Schreibweise) aus schema_postgres.sql.

```
def _lade_schema_spalten() -> dict[str, str]:
    import re

    text = SCHEMA_PATH_POSTGRES.read_text(encoding="utf-8")
    namen = re.findall(r"^\s*(\w+)\s+(?:TEXT|REAL|INTEGER|SERIAL)\b", text, re.MULTILINE)
    return {n.lower(): n for n in namen}
```

Modulkopf / Moduldaten

```
_SCHEMA_SPALTEN = _lade_schema_spalten()
```

def _cursor(...)

Liefert einen Cursor mit dict-artigem Zeilenzugriff (row["Spalte"]).

```
def _cursor(conn):
    """Cursor mit dict-artigem Zeilenzugriff (row["Spalte"]) - bei SQLite
    automatisch ueber row_factory, bei Postgres explizit ueber RealDictCursor.
    Nicht fuer fetch_df verwenden (siehe get_connection)."""
```

```

if _ist_postgres():
    return conn.cursor(cursor_factory=psycopg2.extras.RealDictCursor)
return conn.cursor()

```

Modulkopf / Moduldaten

```

# -----
# Generische Hilfsfunktionen
# -----

```

def fetch_all(...)

Fuehrt eine SELECT-Abfrage aus, liefert alle Zeilen als Liste.

```

def fetch_all(query: str, params: tuple = ()) -> list:
    with connection() as conn:
        cur = _cursor(conn)
        cur.execute(_uebersetze(query), params)
        return [_normalisiere_zeile(r) for r in cur.fetchall()]

```

def fetch_one(...)

Fuehrt eine SELECT-Abfrage aus, liefert die erste Zeile (oder None).

```

def fetch_one(query: str, params: tuple = ()):
    with connection() as conn:
        cur = _cursor(conn)
        cur.execute(_uebersetze(query), params)
        return _normalisiere_zeile(cur.fetchone())

```

def fetch_df(...)

Wie fetch_all, aber direkt als pandas-DataFrame - fuer die Streamlit-Tabellen.

```

def fetch_df(query: str, params: tuple = ()) -> pd.DataFrame:
    """Wie fetch_all, aber direkt als DataFrame - praktisch fuer Streamlit-Tabellen."""
    with connection() as conn:
        df = pd.read_sql_query(_uebersetze(query), conn, params=params)
        return _normalisiere_df(df)

```

def execute(...)

Fuehrt INSERT/UPDATE/DELETE aus, liefert die Anzahl betroffener Zeilen.

```

def execute(query: str, params: tuple = ()) -> int:
    """Fuehrt INSERT/UPDATE/DELETE aus und gibt die Anzahl betroffener Zeilen zurueck."""
    with connection() as conn:
        cur = conn.cursor()
        cur.execute(_uebersetze(query), params)
        return cur.rowcount

```

def value_exists(...)

Prueft, ob ein Wert in einer Spalte schon existiert (Duplikat-Check).

```

def value_exists(table: str, column: str, value: str) -> bool:
    """Entspricht der alten VBA-Funktion WertExistiertInTabelle."""
    row = fetch_one(f"SELECT 1 FROM {table} WHERE {column} = ? LIMIT 1", (value,))
    return row is not None

```

Modulkopf / Moduldaten

```

# -----
# Konfiguration (einstellbare Schwellenwerte etc.)
# -----

```

def hole_konfiguration(...)

Liest einen konfigurierbaren Wert (z.B. Schrott-Grenze), mit Standardwert.

```

def hole_konfiguration(schluesel: str, standard: float) -> float:
    """Liest einen konfigurierbaren Wert (z.B. 'SchrottGrenzeMM'); Standard, falls noch nicht gesetzt."""
    row = fetch_one("SELECT Wert FROM Konfiguration WHERE Schluesel = ?", (schluesel,))
    return float(row["Wert"]) if row else standard

```

def setze_konfiguration(...)

Speichert/aktualisiert einen konfigurierbaren Wert.

```
def setze_konfiguration(schluessel: str, wert: float) -> None:
    execute(
        "INSERT INTO Konfiguration (Schluessel, Wert) VALUES (?, ?) "
        "ON CONFLICT(Schluessel) DO UPDATE SET Wert = excluded.Wert",
        (schluessel, str(wert)),
    )
```

logic.py

Portierung der VBA-Module E (Profil/Material-Lookup), F (Artikelgruppe), G (Artikel-Stueck + Materialentnahme), H (Suche), B (Export) und C (E-Mail-Entwurf) nach Python. Funktionsnamen orientieren sich bewusst an den VBA-Originalen.

Funktions-Uebersicht

Funktion / Bereich	Zweck
schrott_grenze_mm()	Liest die gemeinsame Laengen-Schwelle fuer Rest-/Schrott-Erkennung.
class LagerFehler	
hole_profil_abkuerzung()	Profil -> Barcode-Kuerzel.
hole_material_abkuerzung()	Material -> Barcode-Kuerzel.
hole_material_gruppe()	Material -> zugehoerige Materialgruppe.
make_barcode_ag()	Generiert den naechsten freien Haupt-Barcode fuer eine Artikelgruppe.
create_artikel_gruppe()	Legt eine neue Artikelgruppe an (prueft Duplikat, vergibt Barcode).
hole_daten_ag()	Laedt die Stammdaten einer Artikelgruppe per Barcode.
_naechste_stueck_nummer()	Ermittelt die naechste freie Stueck-Nummer fuer einen Haupt-Barcode.
make_barcode_as()	Baut den vollstaendigen Stueck-Barcode (Haupt-Barcode + laufende Nummer).
add_artikel_stueck()	Bucht ein neues Stueck (voll oder Rest) zu einer Artikelgruppe ein.
delete_artikel_stueck()	Loescht ein Stueck endgueltig (wird intern vom Papierkorb genutzt).
soft_delete_artikel_stueck()	Verschiebt ein Stueck in den Papierkorb statt es sofort zu loeschen.
papierkorb_wiederherstellen()	Legt ein Papierkorb-Stueck wieder in der Ursprungstabelle an.
papierkorb_endgueltig_loeschen()	Entfernt einen Papierkorb-Eintrag unwiderruflich.
edit_artikel_stueck()	Aktualisiert einzelne Felder eines bestehenden Stuecks.

Funktion / Bereich	Zweck
berechne_restlaenge()	Rest = urspruengliche Laenge - genutzte Laenge (mit Plausibilitaetspruefung).
ist_schrott_kandidat()	Prueft, ob eine Restlaenge unterhalb der Schrott-Schwelle liegt.
process_material_entnahme()	Herzstueck der Materialentnahme: bucht Verbrauch, kuerzt/loescht das Ursprungsstueck, legt bei Bedarf ein Reststueck an.
_buche_reststueck_ein()	Legt ein Reststueck an oder erhoehrt die Menge eines identischen bestehenden.
_schreibe_verlauf()	Schreibt einen Protokoll-Eintrag in die Tabelle Verlauf.
suche()	Einfache Suche (Barcode oder Bezeichnung) ueber Artikel_Liste und Reste.
distinct_werte()	Eindeutige Werte einer Spalte ueber Artikel_Liste + Reste, fuer Dropdowns.
erweiterte_suche()	Mehrkriterien-Filter (UND-verknuepft) ueber Artikel_Liste und Reste.
export_excel_bytes()	Baut eine .xlsx-Datei mit einem Blatt je ausgewaehlter Tabelle.
_pdf_tabellen_seite()	Rendert eine Tabelle als eigene Seite in ein bestehendes FPDF-Objekt.
export_pdf_bytes()	Baut eine Querformat-PDF mit einer Seite je ausgewaehlter Tabelle.
export_pdf_bytes_df()	Wie export_pdf_bytes, aber fuer ein einzelnes bereits geladenes DataFrame.
hole_email_betreff()	Baut den E-Mail-Betreff fuer den Tabellen-Export.
hole_email_text()	Baut den Standard-E-Mail-Text fuer den Tabellen-Export.

Vollstaendiger Quellcode

Modulkopf / Moduldaten

"""Geschaeftslogik des Lager Systems.

Portierung der VBA-Module F (Artikelgruppe), G (Artikel-Stueck + Entnahme) und E (Profil/Material-Lookup) nach Python. Die Funktionsnamen orientieren sich bewusst an den Originalen, damit der Vergleich mit dem alten VBA-Code leicht faellt.

"""

from __future__ import annotations

import datetime as _dt

import re

import threading

import pandas as pd

import db

SCHROTT_GRENZE_STANDARD_MM = 1000 # Fallback, falls noch nicht in den Einstellungen gesetzt

def schrott_grenze_mm(...)

Liest die gemeinsame Laengen-Schwelle fuer Rest-/Schrott-Erkennung.

def schrott_grenze_mm() -> float:

"""EINE gemeinsame, einstellbare Laengen-Schwelle (mm) fuer zwei Stellen:

1. Bei einer Entnahme: liegt die Restlaenge darunter, wird nachgefragt / landet sie in 'Reste'.
2. Beim manuellen Hinzufuegen neuer Ware: liegt die Laenge darunter, gilt das Stueck

```

        automatisch als 'Rest' statt als voller Artikel.
        Einstellbar unter Einstellungen -> Schwellenwerte."""
        return db.hole_konfiguration("SchrottGrenzeMM", SCHROTT_GRENZE_STANDARD_MM)

```

Modulkopf / Moduldaten

```

# Schuetzt "Hoechste Nummer suchen + neuen Barcode einfuegen" vor Race-Conditions,
# wenn mehrere Nutzer im Netzwerk gleichzeitig auf denselben Streamlit-Prozess zugreifen.
_SCHREIB_LOCK = threading.Lock()

```

class LagerFehler

```

class LagerFehler(Exception):
    """Fachliche Fehler (z.B. Duplikat, nicht gefunden) - werden in der UI als Meldung gezeigt."""

```

Modulkopf / Moduldaten

```

# -----
# E-Module: Profil / Material Lookups
# -----

```

def hole_profil_abkuerzung(...)

Profil -> Barcode-Kuerzel.

```

def hole_profil_abkuerzung(profil: str) -> str:
    row = db.fetch_one("SELECT Abkuerzung FROM Profil_Abk WHERE Profil = ?", (profil,))
    return row["Abkuerzung"] if row else "XXX"

```

def hole_material_abkuerzung(...)

Material -> Barcode-Kuerzel.

```

def hole_material_abkuerzung(material: str) -> str:
    row = db.fetch_one("SELECT Abkuerzung FROM Material_Gruppe WHERE Material = ?", (material,))
    return row["Abkuerzung"] if row else "XXX"

```

def hole_material_gruppe(...)

Material -> zugehoerige Materialgruppe.

```

def hole_material_gruppe(material: str) -> str:
    row = db.fetch_one("SELECT Gruppe FROM Material_Gruppe WHERE Material = ?", (material,))
    return row["Gruppe"] if row else ""

```

Modulkopf / Moduldaten

```

# -----
# F-Module: Artikelgruppe (Stammdaten, Haupt-Barcode)
# -----

```

def make_barcode_ag(...)

Generiert den naechsten freien Haupt-Barcode fuer eine Artikelgruppe.

```

def make_barcode_ag(profil: str, mass: str, material: str) -> str:
    """Entspricht MakeBarcode_AG: Praefix (Profil+Material-Kuerzel) + naechste 4-stellige Nummer."""
    prefix = hole_profil_abkuerzung(profil) + hole_material_abkuerzung(material)
    rows = db.fetch_all("SELECT Barcode FROM Artikel_Gruppe WHERE Barcode LIKE ?", (prefix + "%",))
    max_number = 0
    for row in rows:
        suffix = row["Barcode"][len(prefix):]
        if suffix.isdigit():
            max_number = max(max_number, int(suffix))
    return f"{prefix}{max_number + 1:04d}"

```

def create_artikel_gruppe(...)

Legt eine neue Artikelgruppe an (prueft Duplikat, vergibt Barcode).

```

def create_artikel_gruppe(profil: str, mass: str, material: str) -> str:
    """Entspricht MakeArtikelGruppe: prueft Duplikat, generiert Barcode, legt Zeile an."""
    bezeichnung = f"{profil}-{mass}-{material}"
    with _SCHREIB_LOCK:
        if db.value_exists("Artikel_Gruppe", "Bezeichnung", bezeichnung):
            raise LagerFehler(f"Dieser Artikel existiert bereits: {bezeichnung}")

    barcode = make_barcode_ag(profil, mass, material)

```

```

        db.execute(
            "INSERT INTO Artikel_Gruppe (Barcode, Bezeichnung, Profil, Mass, Material) VALUES (?, ?, ?, ?,
■ ?)",
            (barcode, bezeichnung, profil, mass, material),
        )
    return barcode

```

def hole_daten_ag(...)

Laedt die Stammdaten einer Artikelgruppe per Barcode.

```

def hole_daten_ag(barcode: str) -> dict | None:
    row = db.fetch_one("SELECT * FROM Artikel_Gruppe WHERE Barcode = ?", (barcode,))
    return dict(row) if row else None

```

Modulkopf / Moduldaten

```

# -----
# G-Module: Artikel-Stueck (Zugang, Bearbeiten, Loeschen)
# -----

```

def _naechste_stueck_nummer(...)

Ermittelt die naechste freie Stueck-Nummer fuer einen Haupt-Barcode.

```

def _naechste_stueck_nummer(table: str, haupt_barcode: str) -> int:
    """Entspricht MakeBarcode_AS: sucht die hoechste Stuecknummer fuer diesen Haupt-Barcode
    INNERHALB der jeweiligen Tabelle (Artikel_Liste und Reste zaehlen unabhaengig, wie im Original)."""
    rows = db.fetch_all(f"SELECT Barcode2 FROM {table} WHERE Barcode = ?", (haupt_barcode,))
    max_number = 0
    for row in rows:
        match = re.search(r"-(\d+)$", row["Barcode2"])
        if match:
            max_number = max(max_number, int(match.group(1)))
    return max_number + 1

```

def make_barcode_as(...)

Baut den vollstaendigen Stueck-Barcode (Haupt-Barcode + laufende Nummer).

```

def make_barcode_as(table: str, haupt_barcode: str) -> str:
    return f"{haupt_barcode}-{_naechste_stueck_nummer(table, haupt_barcode)}"

```

def add_artikel_stueck(...)

Bucht ein neues Stueck (voll oder Rest) zu einer Artikelgruppe ein.

```

def add_artikel_stueck(
    table: str,
    haupt_barcode: str,
    ort: str,
    typ: str,
    laenge_mm: float,
    menge: int,
    e_preis: float,
    e_preis_einheit: str,
) -> str:
    """Entspricht MakeNew_ArtikelStueck: holt Stammdaten aus Artikel_Gruppe, legt neues Stueck an."""
    ag = hole_daten_ag(haupt_barcode)
    if ag is None:
        raise LagerFehler(f"Der Barcode '{haupt_barcode}' wurde in der Artikel_Gruppe nicht gefunden!")

    with _SCHREIB_LOCK:
        barcode2 = make_barcode_as(table, haupt_barcode)
        datum = _dt.datetime.now().isoformat(timespec="seconds")

        db.execute(
            f"""INSERT INTO {table}
            (Datum, Barcode, Barcode2, Bezeichnung, Profil, Mass, LaengeMM,
            MaterialGruppe, Material, OrtRegal, Typ, MengeStueck, EPreis, EPreisEinheit)
            VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)""",
            (
                datum, haupt_barcode, barcode2, ag["Bezeichnung"], ag["Profil"], ag["Mass"],
                laenge_mm or None, hole_material_gruppe(ag["Material"]), ag["Material"],
                ort, typ, menge, e_preis, e_preis_einheit,
            ),
        )
    _schreibe_verlauf(
        haupt_barcode, barcode2, ag["Bezeichnung"], ag["Profil"], ag["Mass"],
        laenge_mm or 0, hole_material_gruppe(ag["Material"]), ag["Material"],
        ort, typ, menge, e_preis, e_preis_einheit,
    )

```

```
)
return barcode2
```

def delete_artikel_stueck(...)

Loescht ein Stueck endgueltig (wird intern vom Papierkorb genutzt).

```
def delete_artikel_stueck(table: str, barcode2: str) -> None:
    row = db.fetch_one(f"SELECT ID FROM {table} WHERE Barcode2 = ?", (barcode2,))
    if row is None:
        raise LagerFehler(f"Artikel mit Barcode '{barcode2}' nicht gefunden.")
    db.execute(f"DELETE FROM {table} WHERE Barcode2 = ?", (barcode2,))
```

Modulkopf / Moduldaten

```
# -----
# Papierkorb: "Loeschen" in der UI verschiebt ein Stueck hierher statt es
# sofort zu entfernen - kann von hier wiederhergestellt oder endgueltig
# geloescht werden.
# -----
```

def soft_delete_artikel_stueck(...)

Verschiebt ein Stueck in den Papierkorb statt es sofort zu loeschen.

```
def soft_delete_artikel_stueck(table: str, barcode2: str) -> None:
    """Verschiebt ein Stueck in den Papierkorb, statt es sofort zu loeschen."""
    zeile = db.fetch_one(f"SELECT * FROM {table} WHERE Barcode2 = ?", (barcode2,))
    if zeile is None:
        raise LagerFehler(f"Artikel mit Barcode '{barcode2}' nicht gefunden.")

    geloescht_am = _dt.datetime.now().isoformat(timespec="seconds")
    db.execute(
        f"""INSERT INTO Papierkorb
        (QuellTabelle, GeloeschtAm, Datum, Barcode, Barcode2, Bezeichnung, Profil, Mass, LaengeMM,
        MaterialGruppe, Material, OrtRegal, Typ, MengeStueck, EPreis, EPreisEinheit)
        VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)""",
        (
            table, geloescht_am, zeile["Datum"], zeile["Barcode"], zeile["Barcode2"], zeile["Bezeichnung"],
            zeile["Profil"], zeile["Mass"], zeile["LaengeMM"], zeile["MaterialGruppe"], zeile["Material"],
            zeile["OrtRegal"], zeile["Typ"], zeile["MengeStueck"], zeile["EPreis"], zeile["EPreisEinheit"],
        ),
    )
    db.execute(f"DELETE FROM {table} WHERE Barcode2 = ?", (barcode2,))
```

def papierkorb_wiederherstellen(...)

Legt ein Papierkorb-Stueck wieder in der Ursprungstabelle an.

```
def papierkorb_wiederherstellen(papierkorb_id: int) -> None:
    """Legt ein Papierkorb-Stueck wieder in seiner urspruenglichen Tabelle an."""
    zeile = db.fetch_one("SELECT * FROM Papierkorb WHERE ID = ?", (papierkorb_id,))
    if zeile is None:
        raise LagerFehler("Eintrag im Papierkorb nicht gefunden.")

    ziel_table = zeile["QuellTabelle"]
    if db.value_exists(ziel_table, "Barcode2", zeile["Barcode2"]):
        raise LagerFehler(
            f"'{zeile['Barcode2']}' existiert bereits wieder in {ziel_table.replace('_', ' ')} - "
            "nicht wiederhergestellt."
        )

    db.execute(
        f"""INSERT INTO {ziel_table}
        (Datum, Barcode, Barcode2, Bezeichnung, Profil, Mass, LaengeMM,
        MaterialGruppe, Material, OrtRegal, Typ, MengeStueck, EPreis, EPreisEinheit)
        VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)""",
        (
            zeile["Datum"], zeile["Barcode"], zeile["Barcode2"], zeile["Bezeichnung"], zeile["Profil"],
            zeile["Mass"], zeile["LaengeMM"], zeile["MaterialGruppe"], zeile["Material"], zeile["OrtRegal"],
            zeile["Typ"], zeile["MengeStueck"], zeile["EPreis"], zeile["EPreisEinheit"],
        ),
    )
    db.execute("DELETE FROM Papierkorb WHERE ID = ?", (papierkorb_id,))
```

def papierkorb_endgueltig_loeschen(...)

Entfernt einen Papierkorb-Eintrag unwiderruflich.

```
def papierkorb_endgueltig_loeschen(papierkorb_id: int) -> None:
    db.execute("DELETE FROM Papierkorb WHERE ID = ?", (papierkorb_id,))
```

def edit_artikel_stueck(...)

Aktualisiert einzelne Felder eines bestehenden Stuecks.

```
def edit_artikel_stueck(table: str, barcode2: str, **felder) -> None:
    """felder z.B. LaengeMM=500, OrtRegal='A1', Typ='Voll', MengeStueck=2, EPreis=1.5,
    ■ EPreisEinheit='Stk'"""
    if not felder:
        return
    if db.fetch_one(f"SELECT ID FROM {table} WHERE Barcode2 = ?", (barcode2,)) is None:
        raise LagerFehler(f"Artikel mit Barcode '{barcode2}' nicht gefunden.")
    zuweisungen = ", ".join(f"{spalte} = ?" for spalte in felder)
    db.execute(f"UPDATE {table} SET {zuweisungen} WHERE Barcode2 = ?", (*felder.values(), barcode2))
```

Modulkopf / Moduldaten

```
# -----
# G7: Materialentnahme-Logik (Kernstueck des Systems)
# -----
```

def berechne_restlaenge(...)

Rest = urspruengliche Laenge - genutzte Laenge (mit Plausibilitaetspruefung).

```
def berechne_restlaenge(initiale_laenge: float, genutzte_laenge: float) -> float:
    rest = initiale_laenge - genutzte_laenge
    if rest < 0:
        raise LagerFehler("Die genutzte Laenge kann nicht groesser als die urspruengliche Laenge sein.")
    return rest
```

def ist_schrott_kandidat(...)

Prueft, ob eine Restlaenge unterhalb der Schrott-Schwelle liegt.

```
def ist_schrott_kandidat(restlaenge: float) -> bool:
    """Entspricht der VBA-Schrott-Abfrage: Reststueck ist zu klein zum sinnvollen Weiterverwenden."""
    return 0 < restlaenge < schrott_grenze_mm()
```

def process_material_entnahme(...)

Herzstueck der Materialentnahme: bucht Verbrauch, kuerzt/loescht das Ursprungsstueck, legt bei Bedarf ein Reststueck an.

```
def process_material_entnahme(
    source_table: str,
    ursprungs_barcode2: str,
    genutzte_laenge: float,
    ist_schrott: bool,
) -> str:
    """Entspricht ProzessMaterialEntnahme: bucht eine Entnahme, reduziert/loescht das Ursprungsstueck,
    legt bei Bedarf ein Reststueck an (in "Reste" bei < 1000mm, sonst zurueck in "Artikel_Liste")."""
    with _SCHREIB_LOCK:
        orig = db.fetch_one(f"SELECT * FROM {source_table} WHERE Barcode2 = ?", (ursprungs_barcode2,))
        if orig is None:
            raise LagerFehler(f"Ursprungsstueck '{ursprungs_barcode2}' nicht gefunden.")

        initiale_laenge = orig["LaengeMM"] or 0
        restlaenge = berechne_restlaenge(initiale_laenge, genutzte_laenge)

        # 1. Verlauf-Eintrag VOR dem Reduzieren (negative Laenge = Verbrauch)
        _schreibe_verlauf(
            orig["Barcode"], orig["Barcode2"], orig["Bezeichnung"], orig["Profil"], orig["Mass"],
            -genutzte_laenge, orig["MaterialGruppe"], orig["Material"], orig["OrtRegal"],
            "Verbrauch", 1, orig["EPreis"], orig["EPreisEinheit"],
        )

        # 2. Ursprungsstueck reduzieren oder loeschen
        if (orig["MengeStueck"] or 1) > 1:
            db.execute(
                f"UPDATE {source_table} SET MengeStueck = MengeStueck - 1 WHERE Barcode2 = ?",
                (ursprungs_barcode2,),
            )
        else:
            db.execute(f"DELETE FROM {source_table} WHERE Barcode2 = ?", (ursprungs_barcode2,))

        # 3. Reststueck einbuchen, falls vorhanden und kein Schrott
        if restlaenge > 0 and not ist_schrott:
            ziel_table = "Reste" if restlaenge < schrott_grenze_mm() else "Artikel_Liste"
            _buche_reststueck_ein(ziel_table, orig, restlaenge)

    return f"Materialentnahme erfolgreich verbucht. Restlaenge: {restlaenge:.0f} mm" if restlaenge > 0 else
```

■ \

"Materialentnahme erfolgreich verbucht (kein Reststueck)."

def _buche_reststueck_ein(...)

Legt ein Reststueck an oder erhoehrt die Menge eines identischen bestehenden.

```
def _buche_reststueck_ein(ziel_table: str, orig: db.sqlite3.Row, neue_laenge: float) -> None:
    """Entspricht BucheRestStueckEin: sucht identisches Reststueck (gleicher Haupt-Barcode + Laenge),
    erhoehrt sonst die Menge, sonst legt es ein neues Stueck mit neuem Barcode2 an."""
    passendes = db.fetch_one(
        f"SELECT ID, MengeStueck FROM {ziel_table} WHERE Barcode = ? AND LaengeMM = ?",
        (orig["Barcode"], neue_laenge),
    )
    if passendes is not None:
        db.execute(
            f"UPDATE {ziel_table} SET MengeStueck = MengeStueck + 1 WHERE ID = ?",
            (passendes["ID"],),
        )
    return

    neuer_barcode2 = make_barcode_as(ziel_table, orig["Barcode"])
    datum = _dt.datetime.now().isoformat(timespec="seconds")
    db.execute(
        f"""INSERT INTO {ziel_table}
        (Datum, Barcode, Barcode2, Bezeichnung, Profil, Mass, LaengeMM,
         MaterialGruppe, Material, OrtRegal, Typ, MengeStueck, EPreis, EPreisEinheit)
        VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, 1, ?, ?)""",
        (
            datum, orig["Barcode"], neuer_barcode2, orig["Bezeichnung"], orig["Profil"], orig["Mass"],
            neue_laenge, orig["MaterialGruppe"], orig["Material"], orig["OrtRegal"],
            "Rest", orig["EPreis"], orig["EPreisEinheit"],
        ),
    )
```

def _schreibe_verlauf(...)

Schreibt einen Protokoll-Eintrag in die Tabelle Verlauf.

```
def _schreibe_verlauf(
    barcode, barcode2, bezeichnung, profil, mass, laenge_mm,
    material_gruppe, material, ort, typ, menge, e_preis, e_preis_einheit,
) -> None:
    datum = _dt.datetime.now().isoformat(timespec="seconds")
    db.execute(
        """INSERT INTO Verlauf
        (Datum, Barcode, Barcode2, Bezeichnung, Profil, Mass, LaengeMM,
         MaterialGruppe, Material, OrtRegal, Typ, MengeStueck, EPreis, EPreisEinheit)
        VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)""",
        (
            datum, barcode, barcode2, bezeichnung, profil, mass, laenge_mm,
            material_gruppe, material, ort, typ, menge, e_preis, e_preis_einheit,
        ),
    )
```

Modulkopf / Moduldaten

```
# -----
# H-Module: Suche
# -----
```

def suche(...)

Einfache Suche (Barcode oder Bezeichnung) ueber Artikel_Liste und Reste.

```
def suche(begriff: str, nach: str = "Barcode") -> dict[str, "db.pd.DataFrame"]:
    """Entspricht Fill_Dashboard_List / BtnSuche_Click: durchsucht Artikel_Liste und Reste.

    Bei nach='Barcode' wird sowohl der Haupt-Barcode als auch der Stueck-Barcode
    (Barcode2) geprueft, damit ein gescannter Stueck-Code (z.B. 'FLS370010-1')
    genauso gefunden wird wie der Gruppen-Code."""
    muster = f"%{begriff}%"
    ergebnisse = {}
    for table in ("Artikel_Liste", "Reste"):
        if nach == "Barcode":
            query = f"SELECT * FROM {table} WHERE Barcode LIKE ? OR Barcode2 LIKE ?"
            params = (muster, muster)
        else:
            query = f"SELECT * FROM {table} WHERE Bezeichnung LIKE ?"
            params = (muster,)
        ergebnisse[table] = db.fetch_df(query, params)
```

```
return ergebnisse
```

def distinct_werte(...)

Eindeutige Werte einer Spalte ueber Artikel_Liste + Reste, fuer Dropdowns.

```
def distinct_werte(spalte: str) -> list[str]:
    """Eindeutige, nicht-leere Werte einer Spalte ueber Artikel_Liste UND Reste - fuer Dropdowns."""
    # Alias bewusst nicht "wert" genannt - kollidiert sonst mit der echten
    # Spalte "Wert" aus der Konfiguration-Tabelle (Postgres behandelt beides
    # gleich, da Bezeichner klein gefaltet werden).
    query = f"""
        SELECT DISTINCT {spalte} AS ergebniswert FROM Artikel_Liste WHERE {spalte} IS NOT NULL AND {spalte}
    """
    if != '':
        UNION
        SELECT DISTINCT {spalte} AS ergebniswert FROM Reste WHERE {spalte} IS NOT NULL AND {spalte} != ''
        ORDER BY ergebniswert
    """
    return [row["ergebniswert"] for row in db.fetch_all(query)]
```

def erweiterte_suche(...)

Mehrkriterien-Filter (UND-verknuepft) ueber Artikel_Liste und Reste.

```
def erweiterte_suche(
    profil: str = "", barcode: str = "", bezeichnung: str = "", ort: str = "",
    material: str = "", mass: str = "", laenge_von: float = 0, laenge_bis: float = 0,
) -> dict[str, "db.pd.DataFrame"]:
    """Filtert Artikel_Liste und Reste ueber mehrere gleichzeitig aktive Kriterien (UND-verknuepft).
    Leere/0-Werte werden ignoriert (kein Filter auf dieses Feld)."""
    ergebnisse = {}
    for table in ("Artikel_Liste", "Reste"):
        bedingungen = []
        params: list = []
        if profil:
            bedingungen.append("Profil = ?")
            params.append(profil)
        if barcode:
            bedingungen.append("(Barcode LIKE ? OR Barcode2 LIKE ?)")
            params += [f"%{barcode}%", f"%{barcode}%"]
        if bezeichnung:
            bedingungen.append("Bezeichnung LIKE ?")
            params.append(f"%{bezeichnung}%")
        if ort:
            bedingungen.append("OrtRegal = ?")
            params.append(ort)
        if material:
            bedingungen.append("Material = ?")
            params.append(material)
        if mass:
            bedingungen.append("Mass = ?")
            params.append(mass)
        if laenge_von:
            bedingungen.append("LaengeMM >= ?")
            params.append(laenge_von)
        if laenge_bis:
            bedingungen.append("LaengeMM <= ?")
            params.append(laenge_bis)

        where = " AND ".join(bedingungen) if bedingungen else "1=1"
        ergebnisse[table] = db.fetch_df(f"SELECT * FROM {table} WHERE {where}", tuple(params))
    return ergebnisse
```

Modulkopf / Moduldaten

```
# -----
# B-Module: Export
# -----
```

```
EXPORTIERBARE_TABELLEN = ["Artikel_Liste", "Reste", "Verlauf", "Artikel_Gruppe"]
```

def export_excel_bytes(...)

Baut eine .xlsx-Datei mit einem Blatt je ausgewaehlter Tabelle.

```
def export_excel_bytes(tabellen: list[str]) -> bytes:
    """Entspricht DashboardExport/ExportiereExcel: baut eine .xlsx-Datei mit einem Blatt
    pro gewaehlter Tabelle und gibt die Rohdaten zurueck (zum Download in der UI)."""
    import io

    puffer = io.BytesIO()
```

```

with pd.ExcelWriter(puffer, engine="openpyxl") as writer:
    for tabelle in tabellen:
        db.fetch_df(f"SELECT * FROM {tabelle}").to_excel(writer, sheet_name=tabelle, index=False)
    return puffer.getvalue()

```

def _pdf_tabellen_seite(...)

Rendert eine Tabelle als eigene Seite in ein bestehendes FPDF-Objekt.

```

def _pdf_tabellen_seite(pdf, titel: str, df: pd.DataFrame) -> None:
    """Rendert eine Tabelle (DataFrame) als eigene Seite in ein bereits geöffnertes FPDF-Objekt."""
    from fpdf.fonts import FontFace

    pdf.add_page()
    pdf.set_font("Helvetica", "B", 14)
    pdf.cell(0, 10, titel, new_x="LMARGIN", new_y="NEXT")

    if df.empty:
        pdf.set_font("Helvetica", "", 10)
        pdf.cell(0, 8, "Keine Einträge vorhanden.")
        return

    pdf.set_font("Helvetica", "", 6)
    spalten = [str(c) for c in df.columns]
    zeilen = df.fillna("").astype(str).values.tolist()
    with pdf.table(headings_style=FontFace(emphasis="BOLD", size_pt=6)) as table:
        table.row(spalten)
        for zeile in zeilen:
            table.row(zeile)

```

def export_pdf_bytes(...)

Baut eine Querformat-PDF mit einer Seite je ausgewählter Tabelle.

```

def export_pdf_bytes(tabellen: list[str]) -> bytes:
    """Entspricht ExportierePDF: eine Querformat-PDF mit einer Seite pro gewählter Tabelle."""
    from fpdf import FPDF

    pdf = FPDF(orientation="L", format="A4")
    pdf.set_auto_page_break(auto=True, margin=10)
    for tabelle in tabellen:
        df = db.fetch_df(f"SELECT * FROM {tabelle}")
        _pdf_tabellen_seite(pdf, tabelle.replace("_", " "), df)
    return bytes(pdf.output())

```

def export_pdf_bytes_df(...)

Wie export_pdf_bytes, aber fuer ein einzelnes bereits geladenes DataFrame.

```

def export_pdf_bytes_df(df: pd.DataFrame, titel: str) -> bytes:
    """Wie export_pdf_bytes, aber fuer eine beliebige bereits geladene Tabelle (z.B. eine
    Einstellungen-Stammdatentabelle oder ein Suchergebnis) - eine einzelne PDF-Seite."""
    from fpdf import FPDF

    pdf = FPDF(orientation="L", format="A4")
    pdf.set_auto_page_break(auto=True, margin=10)
    _pdf_tabellen_seite(pdf, titel, df)
    return bytes(pdf.output())

```

Modulkopf / Moduldaten

```

# -----
# C-Module: Email-Entwurf (mailto, ohne SMTP-Zugangsdaten)
# -----

```

def hole_email_betreff(...)

Baut den E-Mail-Betreff fuer den Tabellen-Export.

```

def hole_email_betreff(tabellen: list[str]) -> str:
    """Entspricht HoleEmailBetreff: 'Lager_System - Stand: DD.MM.YYYY'."""
    if set(tabellen) == set(EXPORTIERBARE_TABELLEN):
        basis = "Lager_System"
    else:
        basis = "+".join(t.replace("_", " ") for t in tabellen)
    return f"{basis} - Stand: {_dt.date.today():%d.%m.%Y}"

```

def hole_email_text(...)

Baut den Standard-E-Mail-Text fuer den Tabellen-Export.

```

def hole_email_text(tabellen: list[str]) -> str:
    """Entspricht HoleEmailText: Standardtext mit Liste der exportierten Tabellen."""
    zeilen = "\n".join(f"- {t}" for t in tabellen)
    jetzt = _dt.datetime.now()
    return (
        "Hallo,\n\n"
        "anbei erhalten Sie die angeforderte(n) Lagerliste(n) als Anhang.\n\n"
        f"Exportierte Tabellen:\n{zeilen}\n\n"
        f"Der Export wurde heute, {jetzt:%d.%m.%Y} um {jetzt:%H:%M} erstellt.\n\n"
        "Bitte die zuvor heruntergeladene Datei manuell an diese E-Mail anhaengen "
        "(Browser koennen das aus Sicherheitsgruenden nicht automatisch tun).\n\n"
        "Mit freundlichen Gruessen"
    )

```

app.py

Ersetzt die Excel-Blaetter Dashboard / Artikel Liste / Reste / Verlauf / Artikel Gruppe / Einstellung durch Seiten dieser App, sowie die UserForms durch modale st.dialog-Fenster.

Funktions-Uebersicht

Funktion / Bereich	Zweck
lookup_liste()	Liest eine einzelne Spalte als einfache Python-Liste (fuer Dropdowns).
segmented_nav()	Wie st.segmented_control, initialisiert den Wert aber sicher vorab.
volle_hoehe()	Berechnet die Pixelhoehe, damit eine Tabelle ohne eigenen Scrollbalken passt.
pdf_download_button()	PDF-Download-Knopf fuer eine beliebige Tabelle.
zeige_tabelle()	Zeigt eine komplette DB-Tabelle inkl. Ueberschrift und PDF-Knopf.
formular_neue_artikelgruppe()	Formular: neue Artikelgruppe anlegen.
formular_neue_materialgruppe()	Formular: neue Materialgruppe anlegen.
formular_hinzufuegen_dialog()	Formular: neues Stueck einbuchen (Voll/Rest wird automatisch erkannt).
_kombinierte_stuecke()	Artikel_Liste + Reste zusammen, mit Quelle-Spalte.
formular_entnahme_dialog()	Formular: Materialentnahme (Laenge abbuchen, Rest automatisch verbuchen).
formular_bearbeiten_dialog()	Formular: bestehendes Stueck bearbeiten oder in den Papierkorb verschieben.
dialog_hinzufuegen()	Modales Fenster fuer den Schnellzugriff 'Hinzufuegen'.
dialog_entnahme()	Modales Fenster fuer den Schnellzugriff 'Material entnehmen'.
dialog_bearbeiten()	Modales Fenster fuer den Schnellzugriff 'Bearbeiten / Loeschen'.
dialog_artikelgruppe()	Modales Fenster fuer 'Neue Artikelgruppe'.
dialog_materialgruppe()	Modales Fenster fuer 'Neue Materialgruppe'.
filter_bar()	Filterleiste fuer eine einzelne Tabellen-Seite (Artikel Liste oder Reste).

Funktion / Bereich	Zweck
page_dashboard()	Seite Dashboard: Kennzahlen, Suche, erweiterte Filter, Export.
page_artikel_liste()	Seite Artikel Liste: Filter + volle Tabelle der vollen Artikel.
page_reste()	Seite Reste: Filter + volle Tabelle der Reststuecke.
page_verlauf()	Seite Verlauf: reine Protokoll-Ansicht, nicht editierbar.
page_papierkorb()	Seite Papierkorb: gelöschte Stuecke wiederherstellen oder endgueltig loeschen.
page_artikel_gruppe()	Seite Artikel Gruppe: neue Gruppen anlegen + Uebersicht bestehender Gruppen.
page_einstellungen()	Seite Einstellungen: Stammdaten (Orte, Profile, Materialien, Einheiten, Typen, Schwellenwerte).

Vollstaendiger Quellcode

Modulkopf / Moduldaten

```

"""Lager System - Streamlit-Oberflaeche.

Ersetzt die Excel-Blaetter Dashboard / Artikel Liste / Reste / Verlauf /
Artikel Gruppe / Einstellung durch Seiten dieser App. Die zugrundeliegende
Logik lebt in logic.py, der Datenzugriff in db.py.
"""
from __future__ import annotations

import urllib.parse

import pandas as pd
import streamlit as st

import db
import logic

st.set_page_config(page_title="Lager System", layout="wide")
db.init_db()

# Zentriert alle st.dialog-Fenster vertikal in der Mitte des Bildschirms
# (Streamlit richtet sie standardmaessig oben aus). Ausserdem Ueberschriften
# (Seitentitel = h1, Abschnitts-Ueberschriften = h2/h3) eine Stufe kleiner.
st.markdown(
    """<style>
    [data-testid="stDialog"] > div { align-items: center !important; }
    h1 { font-size: 22px !important; }
    h2, h3 { font-size: 17px !important; }
    </style>""",
    unsafe_allow_html=True,
)

# -----
# Hilfsfunktionen fuer wiederkehrende UI-Bausteine
# -----

```

def lookup_liste(...)

Liest eine einzelne Spalte als einfache Python-Liste (fuer Dropdowns).

```

def lookup_liste(query: str) -> list[str]:
    return [row[0] for row in db.fetch_all(query)]

```

def segmented_nav(...)

Wie st.segmented_control, initialisiert den Wert aber sicher vorab.

```

def segmented_nav(label: str, optionen: list[str], key: str) -> str:
    """Wie st.segmented_control, initialisiert den Wert aber sicher vor dem ersten Rendern."""

```

```

if key not in st.session_state:
    st.session_state[key] = optionen[0]
return st.segmented_control(label, optionen, key=key)

```

def volle_hoehe(...)

Berechnet die Pixelhoehe, damit eine Tabelle ohne eigenen Scrollbalken passt.

```

def volle_hoehe(df: pd.DataFrame) -> int:
    """Hoehe in Pixeln, die eine st.dataframe braucht, um ALLE Zeilen ohne eigenen
    Scrollbalken zu zeigen - die Seite selbst scrollt dann stattdessen."""
    zeilenhoehe = 35
    kopfhoehe = 38
    puffer = 3
    return kopfhoehe + zeilenhoehe * len(df) + puffer

```

def pdf_download_button(...)

PDF-Download-Knopf fuer eine beliebige Tabelle.

```

def pdf_download_button(df: pd.DataFrame, titel: str, key: str, label: str = "■ Als PDF herunterladen") ->
■ None:
    """PDF-Download-Knopf fuer eine beliebige Tabelle - CSV gibt es bereits automatisch
    ueber das Symbol-Menue oben rechts in jeder st.dataframe (Streamlit-Bordmittel, nicht erweiterbar)."""
    st.download_button(
        label,
        data=logic.export_pdf_bytes_df(df, titel),
        file_name=f"{titel}_{pd.Timestamp.now():%y-%m-%d_%H-%M}.pdf",
        mime="application/pdf",
        key=key,
        use_container_width=True,
    )

```

def zeige_tabelle(...)

Zeigt eine komplette DB-Tabelle inkl. Ueberschrift und PDF-Knopf.

```

def zeige_tabelle(table: str, titel: str, order_by: str | None = None) -> None:
    query = f"SELECT * FROM {table}" + (f" ORDER BY {order_by}" if order_by else "")
    df = db.fetch_df(query)
    col_titel, col_pdf = st.columns([5, 1])
    col_titel.subheader(f"{titel} ({len(df)})")
    if df.empty:
        st.info("Noch keine Eintraege vorhanden.")
    else:
        with col_pdf:
            st.write("") # richtet den Button vertikal auf Hoehe der Ueberschrift aus
            pdf_download_button(df, titel, key=f"pdf_{table}_{titel}", label="■ PDF")
            st.dataframe(df, use_container_width=True, hide_index=True, height=volle_hoehe(df))

```

def formular_neue_artikelgruppe(...)

Formular: neue Artikelgruppe anlegen.

```

def formular_neue_artikelgruppe() -> None:
    profile = lookup_liste("SELECT Profil FROM Profil_Abk ORDER BY Profil")
    materialien = lookup_liste("SELECT Material FROM Material_Gruppe ORDER BY Material")

    if not profile or not materialien:
        st.warning("Bitte zuerst unter 'Einstellungen' mindestens ein Profil und ein Material anlegen.")
        return

    with st.form("form_neue_gruppe", clear_on_submit=True):
        profil = st.selectbox("Profil", profile)
        mass = st.text_input("Mass")
        material = st.selectbox("Material", materialien)
        _, col_btn = st.columns([3, 1])
        abgeschickt = col_btn.form_submit_button("Anlegen", type="primary", use_container_width=True)
        if abgeschickt:
            try:
                barcode = logic.create_artikel_gruppe(profil, mass, material)
                st.success(f"Neue Artikelgruppe angelegt! Barcode: {barcode}")
            except logic.LagerFehler as e:
                st.error(str(e))

```

def formular_neue_materialgruppe(...)

Formular: neue Materialgruppe anlegen.

```

def formular_neue_materialgruppe() -> None:
    with st.form("form_material", clear_on_submit=True):

```

```

material = st.text_input("Material")
gruppe = st.text_input("Gruppe")
abk = st.text_input("Abkuerzung (fuer Barcode)")
_, col_btn = st.columns([3, 1])
abgeschickt = col_btn.form_submit_button("Anlegen", type="primary", use_container_width=True)
if abgeschickt and material and gruppe and abk:
    try:
        db.execute(
            "INSERT INTO Material_Gruppe (Material, Gruppe, Abkuerzung) VALUES (?, ?, ?)",
            (material, gruppe, abk.upper()),
        )
        st.success("Hinzugefuegt.")
    except db.IntegrityError:
        st.error("Existiert bereits.")

```

Modulkopf / Moduldaten

```

# -----
# Kombinierte Formulare fuer den Dashboard-Schnellzugriff:
# EIN Formular fuer Artikel_Liste + Reste zusammen (kein Auswahlbutton mehr noetig).
# Bei "Hinzufuegen" entscheidet die Laenge automatisch ueber das Ziel.
# -----

```

def formular_hinzufuegen_dialog(...)

Formular: neues Stueck einbuchen (Voll/Rest wird automatisch erkannt).

```

def formular_hinzufuegen_dialog() -> None:
    gruppen = db.fetch_df("SELECT Barcode, Bezeichnung FROM Artikel_Gruppe ORDER BY Bezeichnung")
    if gruppen.empty:
        st.warning("Bitte zuerst eine Artikelgruppe anlegen.")
        return

    orte = lookup_liste("SELECT OrtRegal FROM Lager ORDER BY OrtRegal")
    einheiten = lookup_liste("SELECT Symbol FROM Einheit ORDER BY Symbol")

    auswahl = st.selectbox(
        "Artikelgruppe", gruppen["Barcode"] + " - " + gruppen["Bezeichnung"], key="hz_gruppe"
    )
    haupt_barcode = auswahl.split(" - ")[0]
    ort = st.selectbox("Ort/Regal", orte, key="hz_ort") if orte else st.text_input("Ort/Regal",
    key="hz_ort")
    laenge = st.number_input("Laenge/mm", min_value=0.0, step=1.0, key="hz_laenge")
    menge = st.number_input("Menge/Stueck", min_value=1, step=1, value=1, key="hz_menge")
    e_preis = st.number_input("E-Preis", min_value=0.0, step=0.1, format="%.2f", key="hz_preis")
    e_preis_einheit = (
        st.selectbox("E-Preis pro Einheit", einheiten, key="hz_einheit")
        if einheiten else st.text_input("E-Preis pro Einheit", key="hz_einheit")
    )

    grenze = logic.schrott_grenze_mm()
    auto_ist_rest = 0 < laenge <= grenze
    if laenge > 0:
        if auto_ist_rest:
            st.warning(
                f"Laenge {laenge:.0f} mm ≤ {grenze:.0f} mm → wird automatisch als **Rest** gespeichert."
            )
        else:
            st.info(
                f"Laenge {laenge:.0f} mm > {grenze:.0f} mm → wird als **voller Artikel** gespeichert."
            )

    ueberschreiben = st.checkbox("Automatische Zuordnung manuell aendern", key="hz_override")
    ziel_ist_rest = auto_ist_rest
    if ueberschreiben:
        ziel_label = st.radio(
            "Speichern als", ["Voller Artikel", "Rest"],
            index=1 if auto_ist_rest else 0, key="hz_zielwahl", horizontal=True,
        )
        ziel_ist_rest = ziel_label == "Rest"

    ziel_table = "Reste" if ziel_ist_rest else "Artikel_Liste"
    typ = "Rest" if ziel_ist_rest else "Voll"

    _, col_btn = st.columns([3, 1])
    if col_btn.button("Hinzufuegen", type="primary", key="hz_submit", use_container_width=True):
        try:
            neuer_barcode = logic.add_artikel_stueck(
                ziel_table, haupt_barcode, ort, typ, laenge, int(menge), e_preis, e_preis_einheit
            )
            st.success(f"Als {typ} eingebucht! Stueck-Barcode: {neuer_barcode}")
            for k in ("hz_laenge", "hz_menge", "hz_preis", "hz_override", "hz_zielwahl"):

```

```

        st.session_state.pop(k, None)
    st.rerun()
except logic.LagerFehler as e:
    st.error(str(e))

```

def _kombinierte_stuecke(...)

Artikel_Liste + Reste zusammen, mit Quelle-Spalte.

```

def _kombinierte_stuecke(spalten: str) -> pd.DataFrame:
    """Artikel_Liste + Reste zusammen, mit einer 'Quelle'-Spalte, die die Herkunftstabelle festhaelt."""
    al = db.fetch_df(f"SELECT {spalten} FROM Artikel_Liste")
    al["Quelle"] = "Artikel_Liste"
    re_df = db.fetch_df(f"SELECT {spalten} FROM Reste")
    re_df["Quelle"] = "Reste"
    return pd.concat([al, re_df], ignore_index=True)

```

def formular_entnahme_dialog(...)

Formular: Materialentnahme (Laenge abbuchen, Rest automatisch verbuchen).

```

def formular_entnahme_dialog() -> None:
    stuecke = _kombinierte_stuecke("Barcode2, Bezeichnung, LaengeMM, MengeStueck")
    if stuecke.empty:
        st.info("Keine Artikel zum Entnehmen vorhanden.")
        return

    optionen = (
        stuecke["Barcode2"] + " - " + stuecke["Bezeichnung"].fillna("")
        + " (" + stuecke["LaengeMM"].fillna(0).astype(str) + " mm, " + stuecke["Quelle"] + ")")
    )
    auswahl = st.selectbox("Stueck waehlen", optionen, key="ent_auswahl")
    barcode2 = auswahl.split(" - ")[0]

    zeile = stuecke[stuecke["Barcode2"] == barcode2].iloc[0]
    table = zeile["Quelle"]
    initiale_laenge = float(zeile["LaengeMM"] or 0)

    # Eigener Key je Stueck (statt fixer Key) - so zeigen die Felder beim Wechsel der
    # Auswahl garantiert den frischen Wert, ganz ohne session_state manuell zuruecksetzen.
    st.caption(f"Urspruengliche Laenge: {initiale_laenge:.0f} mm ({table.replace('_', ' ')})")
    genutzt = st.number_input(
        "Wie viel wurde abgeschnitten/genutzt (mm)?", min_value=0.0, max_value=initiale_laenge,
        step=1.0, key=f"ent_genutzt_{barcode2}",
    )
    rest = initiale_laenge - genutzt
    st.caption(f"Berechnete Restlaenge: {rest:.0f} mm")

    ist_schrott = False
    if logic.ist_schrott_kandidat(rest):
        ist_schrott = st.checkbox(
            f"Reststueck ist nur {rest:.0f} mm lang - als Schrott/Muell entsorgen "
            "(statt in die Reste-Liste zu legen)?",
            key=f"ent_schrott_{barcode2}",
        )

    _, col_btn = st.columns([3, 1])
    if col_btn.button("Entnehmen", type="primary", key="ent_submit", use_container_width=True):
        try:
            msg = logic.process_material_entnahme(table, barcode2, genutzt, ist_schrott)
            st.success(msg)
            st.rerun()
        except logic.LagerFehler as e:
            st.error(str(e))

```

def formular_bearbeiten_dialog(...)

Formular: bestehendes Stueck bearbeiten oder in den Papierkorb verschieben.

```

def formular_bearbeiten_dialog() -> None:
    stuecke = _kombinierte_stuecke("Barcode2, Bezeichnung, LaengeMM, OrtRegal, MengeStueck, EPreis,
    ■ EPreisEinheit")
    if stuecke.empty:
        st.info("Keine Artikel vorhanden.")
        return

    barcode2 = st.selectbox("Stueck waehlen", stuecke["Barcode2"], key="be_auswahl")
    zeile = stuecke[stuecke["Barcode2"] == barcode2].iloc[0]
    table = zeile["Quelle"]

    orte = lookup_liste("SELECT OrtRegal FROM Lager ORDER BY OrtRegal")
    einheiten = lookup_liste("SELECT Symbol FROM Einheit ORDER BY Symbol")

```

```

# Eigener Key je Stueck (statt fixer Key) - so zeigen die Felder beim Wechsel der
# Auswahl garantiert den frischen Wert, ganz ohne session_state manuell zuruecksetzen.
neue_laenge = st.number_input("Laenge/mm", value=float(zeile["LaengeMM"] or 0), min_value=0.0,
■ key=f"be_laenge_{barcode2}")
    neuer_ort = (
        st.selectbox(
            "Ort/Regal", orte,
            index=orte.index(zeile["OrtRegal"]) if zeile["OrtRegal"] in orte else 0, key=f"be_ort_{
■ barcode2}",
        ) if orte else st.text_input("Ort/Regal", value=zeile["OrtRegal"] or "", key=f"be_ort_{barcode2}")
    )
    neue_menge = st.number_input("Menge/Stueck", value=int(zeile["MengeStueck"] or 1), min_value=1, step=1,
■ key=f"be_menge_{barcode2}")
    neuer_preis = st.number_input("E-Preis", value=float(zeile["EPreis"] or 0), min_value=0.0, format="%.
■ 2f", key=f"be_preis_{barcode2}")
    neue_einheit = (
        st.selectbox(
            "E-Preis pro Einheit", einheiten,
            index=einheiten.index(zeile["EPreisEinheit"]) if zeile["EPreisEinheit"] in einheiten else 0,
■ key=f"be_einheit_{barcode2}",
        ) if einheiten else st.text_input("E-Preis pro Einheit", value=zeile["EPreisEinheit"] or "",
■ key=f"be_einheit_{barcode2}")
    )

confirm_key = f"be_confirm_del_{barcode2}"
col_del, _, col_save = st.columns([1, 2, 1])
if col_del.button("Loeschen", key=f"be_del_{barcode2}"):
    st.session_state[confirm_key] = True
if col_save.button("Speichern", type="primary", key=f"be_save_{barcode2}", use_container_width=True):
    logic.edit_artikel_stueck(
        table, barcode2, LaengeMM=neue_laenge, OrtRegal=neuer_ort,
        MengeStueck=int(neue_menge), EPreis=neuer_preis, EPreisEinheit=neue_einheit,
    )
    st.success("Aenderungen gespeichert.")
    st.rerun()

if st.session_state.get(confirm_key):
    st.warning(f"'{barcode2}' wirklich loeschen? Das Stueck wandert in den Papierkorb.")
    col_ja, col_abbrechen = st.columns(2)
    if col_ja.button("Ja, loeschen", type="primary", key=f"be_del_ja_{barcode2}",
■ use_container_width=True):
        logic.soft_delete_artikel_stueck(table, barcode2)
        st.session_state.pop(confirm_key, None)
        st.warning(f"Artikel '{barcode2}' in den Papierkorb verschoben.")
        st.rerun()
    if col_abbrechen.button("Abbrechen", key=f"be_del_abbrechen_{barcode2}", use_container_width=True):
        st.session_state.pop(confirm_key, None)
        st.rerun()

```

Modulkopf / Moduldaten

```

# -----
# Dialoge fuer den Dashboard-Schnellzugriff (modale Fenster statt Seitenwechsel)
# -----

```

def dialog_hinzufuegen(...)

Modales Fenster fuer den Schnellzugriff 'Hinzufuegen'.

```

@st.dialog("Hinzufuegen", width="medium")
def dialog_hinzufuegen() -> None:
    formular_hinzufuegen_dialog()

```

def dialog_entnahme(...)

Modales Fenster fuer den Schnellzugriff 'Material entnehmen'.

```

@st.dialog("Material entnehmen", width="medium")
def dialog_entnahme() -> None:
    formular_entnahme_dialog()

```

def dialog_bearbeiten(...)

Modales Fenster fuer den Schnellzugriff 'Bearbeiten / Loeschen'.

```

@st.dialog("Bearbeiten / Loeschen", width="medium")
def dialog_bearbeiten() -> None:
    formular_bearbeiten_dialog()

```

def dialog_artikel_gruppe(...)

Modales Fenster fuer 'Neue Artikelgruppe'.

```
@st.dialog("Neue Artikelgruppe")
def dialog_artikel_gruppe() -> None:
    formular_neue_artikelgruppe()
```

def dialog_material_gruppe(...)

Modales Fenster fuer 'Neue Materialgruppe'.

```
@st.dialog("Neue Materialgruppe")
def dialog_material_gruppe() -> None:
    formular_neue_materialgruppe()
```

Modulkopf / Moduldaten

```
# -----
# Filterleiste fuer die Uebersicht einer einzelnen Tabelle (Artikel_Liste
# oder Reste) - anders als die "Erweiterte Filter" im Dashboard nur auf
# diese eine Tabelle bezogen.
# -----
```

def filter_bar(...)

Filterleiste fuer eine einzelne Tabellen-Seite (Artikel Liste oder Reste).

```
def filter_bar(table: str, key_prefix: str) -> pd.DataFrame | None:
    """Zeigt eine Filterleiste fuer 'table'. Gibt das gefilterte DataFrame zurueck,
    sobald gefiltert wurde - sonst None (dann zeigt die Seite die volle Tabelle)."""
    profil_opt = [""] + lookup_liste("SELECT DISTINCT Profil FROM Artikel_Gruppe ORDER BY Profil")
    material_opt = [""] + lookup_liste("SELECT DISTINCT Material FROM Artikel_Gruppe ORDER BY Material")
    ort_opt = [""] + lookup_liste("SELECT OrtRegal FROM Lager ORDER BY OrtRegal")
    barcode_opt = lookup_liste(f"SELECT DISTINCT Barcode2 FROM {table} ORDER BY Barcode2")
    bezeichnung_opt = lookup_liste(f"SELECT DISTINCT Bezeichnung FROM {table} ORDER BY Bezeichnung")
    mass_opt = lookup_liste(f"SELECT DISTINCT Mass FROM {table} ORDER BY Mass")

    with st.form(f"{key_prefix}_filter_form"):
        c1, c2, c3 = st.columns(3)
        f_profil = c1.selectbox("Profil", profil_opt, key=f"{key_prefix}_f_profil")
        f_material = c2.selectbox("Material", material_opt, key=f"{key_prefix}_f_material")
        f_ort = c3.selectbox("Lagerort", ort_opt, key=f"{key_prefix}_f_ort")

        c4, c5, c6 = st.columns(3)
        f_barcode = c4.selectbox(
            "Barcode", barcode_opt, index=None, accept_new_options=True,
            placeholder="Barcode...", key=f"{key_prefix}_f_barcode",
        )
        f_bezeichnung = c5.selectbox(
            "Bezeichnung", bezeichnung_opt, index=None, accept_new_options=True,
            placeholder="Bezeichnung...", key=f"{key_prefix}_f_bezeichnung",
        )
        f_mass = c6.selectbox(
            "Mass", mass_opt, index=None, accept_new_options=True,
            placeholder="Mass...", key=f"{key_prefix}_f_mass",
        )

        c7, c8 = st.columns(2)
        f_laenge_von = c7.number_input("Laenge von (mm)", min_value=0.0, step=10.0, key=f"{
        key_prefix}_f_laenge_von")
        f_laenge_bis = c8.number_input("Laenge bis (mm)", min_value=0.0, step=10.0, key=f"{
        key_prefix}_f_laenge_bis")

        _, col_btn = st.columns([3, 1])
        gefiltert = col_btn.form_submit_button("Filtern", type="primary", use_container_width=True)

        if not gefiltert:
            return None

        ergebnisse = logic.erweiterte_suche(
            profil=f_profil or "", barcode=f_barcode or "", bezeichnung=f_bezeichnung or "",
            ort=f_ort or "", material=f_material or "", mass=f_mass or "",
            laenge_von=f_laenge_von, laenge_bis=f_laenge_bis,
        )
        return ergebnisse.get(table, pd.DataFrame())
```

Modulkopf / Moduldaten

```
# -----
```

Seiten

def page_dashboard(...)

Seite Dashboard: Kennzahlen, Suche, erweiterte Filter, Export.

```
def page_dashboard() -> None:
    st.title("Dashboard")

    col1, col2, col3 = st.columns(3)
    col1.metric("Artikel (voll)", db.fetch_one("SELECT COUNT(*) c FROM Artikel_Liste")["c"])
    col2.metric("Reststuecke", db.fetch_one("SELECT COUNT(*) c FROM Reste")["c"])
    col3.metric("Artikelgruppen", db.fetch_one("SELECT COUNT(*) c FROM Artikel_Gruppe")["c"])

    st.divider()
    st.subheader("Suche")
    col_nach, col_begriff = st.columns([1, 3])
    nach = col_nach.radio("Suchen nach", ["Barcode", "Bezeichnung"], horizontal=True)

    if nach == "Barcode":
        vorschlaege = sorted(set(logic.distinct_werte("Barcode")) | set(logic.distinct_werte("Barcode2")))
    else:
        vorschlaege = logic.distinct_werte("Bezeichnung")

    begriff = col_begriff.selectbox(
        "Suchbegriff",
        options=vorschlaege,
        index=None,
        accept_new_options=True,
        placeholder="Aus Liste waehlen, tippen oder Barcode scannen...",
    )

    if begriff:
        ergebnisse = logic.suche(begriff, nach)
        for name, df in ergebnisse.items():
            if df.empty:
                st.write(f"***{name}** ({len(df)} Treffer)")
                continue
            col_titel, col_pdf = st.columns([5, 1])
            col_titel.write(f"***{name}** ({len(df)} Treffer)")
            with col_pdf:
                pdf_download_button(df, f"Suche_{name}", key=f"pdf_suche_{name}", label="■ PDF")
            st.dataframe(df, use_container_width=True, hide_index=True, height=volle_hoehe(df))
            st.caption(f"Summe Laenge x Menge: {(df['LaengeMM'].fillna(0) * df['MengeStueck'].fillna(0)).sum():.0f} mm")

    st.divider()
    st.subheader("Erweiterte Filter")
    with st.form("erweiterte_filter_form"):
        profil_opt = [""] + lookup_liste("SELECT DISTINCT Profil FROM Artikel_Gruppe ORDER BY Profil")
        material_opt = [""] + lookup_liste("SELECT DISTINCT Material FROM Artikel_Gruppe ORDER BY Material")
        ort_opt = [""] + lookup_liste("SELECT OrtRegal FROM Lager ORDER BY OrtRegal")

        c1, c2, c3, c4 = st.columns(4)
        f_profil = c1.selectbox("Profil", profil_opt, key="filter_profil")
        f_material = c2.selectbox("Material", material_opt, key="filter_material")
        f_ort = c3.selectbox("Lagerort", ort_opt, key="filter_ort")
        f_barcode = c4.selectbox(
            "Barcode", options=vorschlaege if nach == "Barcode" else [],
            index=None, accept_new_options=True, placeholder="Barcode...", key="filter_barcode",
        )

        c5, c6, c7, c8 = st.columns(4)
        f_bezeichnung = c5.selectbox(
            "Bezeichnung", options=logic.distinct_werte("Bezeichnung"),
            index=None, accept_new_options=True, placeholder="Bezeichnung...", key="filter_bezeichnung",
        )
        f_mass = c6.selectbox(
            "Mass", options=lookup_liste("SELECT DISTINCT Mass FROM Artikel_Gruppe ORDER BY Mass"),
            index=None, accept_new_options=True, placeholder="Mass...", key="filter_mass",
        )
        f_laenge_von = c7.number_input("Laenge von (mm)", min_value=0.0, step=10.0, key="filter_laenge_von")
        f_laenge_bis = c8.number_input("Laenge bis (mm)", min_value=0.0, step=10.0, key="filter_laenge_bis")

        _, col_btn = st.columns([3, 1])
        gefiltert = col_btn.form_submit_button("Filtern", type="primary", use_container_width=True)

    if gefiltert:
        ergebnisse = logic.erweiterte_suche(
            profil=f_profil or "", barcode=f_barcode or "", bezeichnung=f_bezeichnung or "",
            ort=f_ort or "", material=f_material or "", mass=f_mass or "",
        )
```

```

        laenge_von=f_laenge_von, laenge_bis=f_laenge_bis,
    )
    frames = []
    for quelle, df in ergebnisse.items():
        if not df.empty:
            df = df.copy()
            df.insert(0, "Quelle", quelle)
            frames.append(df)
    if frames:
        kombiniert = pd.concat(frames, ignore_index=True)
        col_titel, col_pdf = st.columns([5, 1])
        col_titel.success(f"{len(kombiniert)} Treffer")
        with col_pdf:
            pdf_download_button(kombiniert, "Erweiterte_Filter", key="pdf_erweiterte_filter", label="■
■ PDF")
            st.dataframe(kombiniert, use_container_width=True, hide_index=True, height=volle_hoehe(
■ kombiniert))
        else:
            st.info("Keine Treffer fuer diese Filterkombination.")

    st.divider()
    st.subheader("Exportieren")
    st.caption("Entspricht dem Export-Button im alten Excel-Dashboard.")
    ausgewaehlt = st.multiselect(
        "Welche Tabellen exportieren?", logic.EXPORTIERBARE_TABELLEN, default=logic.EXPORTIERBARE_TABELLEN
    )
    if ausgewaehlt:
        zeitstempel = f"{pd.Timestamp.now():%y-%m-%d_%H-%M}"
        col_excel, col_pdf = st.columns(2)
        col_excel.download_button(
            "Als Excel herunterladen",
            data=logic.export_excel_bytes(ausgewaehlt),
            file_name=f"Lager_System_{zeitstempel}.xlsx",
            mime="application/vnd.openxmlformats-officedocument.spreadsheetml.sheet",
        )
        col_pdf.download_button(
            "Als PDF herunterladen",
            data=logic.export_pdf_bytes(ausgewaehlt),
            file_name=f"Lager_System_{zeitstempel}.pdf",
            mime="application/pdf",
        )

    st.markdown("***Per E-Mail versenden**")
    st.caption(
        "Oeffnet einen vorausgefüllten Entwurf in deinem Standard-Mail-Programm. "
        "Die oben heruntergeladene Datei bitte manuell anhaengen."
    )
    empfaenger = st.text_input("Empfaenger-Email", key="email_empfaenger")
    if empfaenger:
        betreff = logic.hole_email_betreff(ausgewaehlt)
        text = logic.hole_email_text(ausgewaehlt)
        mailto_link = (
            f"mailto:{urllib.parse.quote(empfaenger)}"
            f"?subject={urllib.parse.quote(betreff)}"
            f"&body={urllib.parse.quote(text)}"
        )
        st.link_button("E-Mail-Entwurf oeffnen", mailto_link)

```

def page_artikel_liste(...)

Seite Artikel Liste: Filter + volle Tabelle der vollen Artikel.

```

def page_artikel_liste() -> None:
    st.title("Artikel Liste")

    st.subheader("Filter")
    gefiltert = filter_bar("Artikel_Liste", "al")

    st.divider()
    if gefiltert is None:
        zeige_tabelle("Artikel_Liste", "Volle Artikel", order_by="ID DESC")
    elif gefiltert.empty:
        st.subheader("Gefiltertes Ergebnis (0)")
        st.info("Keine Treffer fuer diese Filterkombination.")
    else:
        col_titel, col_pdf = st.columns([5, 1])
        col_titel.subheader(f"Gefiltertes Ergebnis ({len(gefiltert)})")
        with col_pdf:
            st.write("")
            pdf_download_button(gefiltert, "Artikel_Liste_gefiltert", key="pdf_al_gefiltert", label="■ PDF")
        st.dataframe(gefiltert, use_container_width=True, hide_index=True, height=volle_hoehe(gefiltert))

```

def page_reste(...)

Seite Reste: Filter + volle Tabelle der Reststuecke.

```
def page_reste() -> None:
    st.title("Reste")

    st.subheader("Filter")
    gefiltert = filter_bar("Reste", "re")

    st.divider()
    if gefiltert is None:
        zeige_tabelle("Reste", "Reststuecke", order_by="ID DESC")
    elif gefiltert.empty:
        st.subheader("Gefiltertes Ergebnis (0)")
        st.info("Keine Treffer fuer diese Filterkombination.")
    else:
        col_titel, col_pdf = st.columns([5, 1])
        col_titel.subheader(f"Gefiltertes Ergebnis ({len(gefiltert)})")
        with col_pdf:
            st.write("")
            pdf_download_button(gefiltert, "Reste_gefiltert", key="pdf_re_gefiltert", label="■ PDF")
        st.dataframe(gefiltert, use_container_width=True, hide_index=True, height=volle_hoehe(gefiltert))
```

def page_verlauf(...)

Seite Verlauf: reine Protokoll-Ansicht, nicht editierbar.

```
def page_verlauf() -> None:
    st.title("Verlauf")
    st.caption("Reine Protokoll-Ansicht - nicht editierbar, wie im urspruenglichen Excel-System.")
    zeige_tabelle("Verlauf", "Alle Buchungen", order_by="ID DESC")
```

def page_papierkorb(...)

Seite Papierkorb: geloeschte Stuecke wiederherstellen oder endgueltig loeschen.

```
def page_papierkorb() -> None:
    st.title("Papierkorb")
    st.caption("Geloeschte Artikel/Reste - koennen wiederhergestellt oder endgueltig entfernt werden.")

    df = db.fetch_df("SELECT * FROM Papierkorb ORDER BY ID DESC")
    if df.empty:
        st.info("Papierkorb ist leer.")
        return

    for _, zeile in df.iterrows():
        eintrag_id = int(zeile["ID"])
        with st.container(border=True):
            col_info, col_wieder, col_loeschen = st.columns([4, 1, 1])
            col_info.write(
                f"*{zeile['Barcode2']}* - {zeile['Bezeichnung']} or '' "
                f"({zeile['LaengeMM']} or 0:0f mm, {str(zeile['QuellTabelle']).replace('_', ' ')}) \n"
                f"Geloescht am {zeile['GeloeschAm']}"
            )
            if col_wieder.button("■ Wiederherstellen", key=f"pk_wieder_{eintrag_id}",
    ■ use_container_width=True):
                try:
                    logic.papierkorb_wiederherstellen(eintrag_id)
                    st.success(f"'{zeile['Barcode2']}' wiederhergestellt.")
                    st.rerun()
                except logic.LagerFehler as e:
                    st.error(str(e))

            confirm_key = f"pk_confirm_del_{eintrag_id}"
            if col_loeschen.button("■ Endgueltig loeschen", key=f"pk_del_{eintrag_id}",
    ■ use_container_width=True):
                st.session_state[confirm_key] = True

            if st.session_state.get(confirm_key):
                st.error(f"'{zeile['Barcode2']}' wirklich UNWIDERRUFLICH loeschen? Das kann nicht
    ■ rueckgaengig gemacht werden.")
                col_ja, col_abbrechen = st.columns(2)
                if col_ja.button("Ja, endgueltig loeschen", type="primary", key=f"pk_ja_{eintrag_id}",
    ■ use_container_width=True):
                    logic.papierkorb_endgueltig_loeschen(eintrag_id)
                    st.session_state.pop(confirm_key, None)
                    st.warning(f"'{zeile['Barcode2']}' endgueltig geloescht.")
                    st.rerun()
                if col_abbrechen.button("Abbrechen", key=f"pk_abbrechen_{eintrag_id}",
    ■ use_container_width=True):
                    st.session_state.pop(confirm_key, None)
```

```
st.rerun()
```

def page_artikel_gruppe(...)

Seite Artikel Gruppe: neue Gruppen anlegen + Uebersicht bestehender Gruppen.

```
def page_artikel_gruppe() -> None:
    st.title("Artikel Gruppe")
    st.subheader("Neue Artikelgruppe anlegen")
    formular_neue_artikelgruppe()
    st.divider()
    zeige_tabelle("Artikel-Gruppe", "Bestehende Artikelgruppen")
```

def page_einstellungen(...)

Seite Einstellungen: Stammdaten (Orte, Profile, Materialien, Einheiten, Typen, Schwellenwerte).

```
def page_einstellungen() -> None:
    st.title("Einstellungen")
    st.caption("Stammdaten fuer Dropdowns in den anderen Seiten - entspricht dem alten Blatt 'Einstellung'."
    st.divider()

    einst_optionen = ["Lager/Orte", "Profil-Kuerzel", "Material-Gruppen", "Einheiten", "Typen",
    "Schwellenwerte"]
    auswahl = segmented_nav("Bereich", einst_optionen, "einst_subnav")

    if auswahl == "Lager/Orte":
        with st.form("form_lager", clear_on_submit=True):
            ort = st.text_input("Neuer Ort/Regal")
            _, col_btn = st.columns([3, 1])
            if col_btn.form_submit_button("Hinzufuegen", use_container_width=True) and ort:
                try:
                    db.execute("INSERT INTO Lager (OrtRegal) VALUES (?)", (ort,))
                    st.success("Hinzugefuegt.")
                except db.IntegrityError:
                    st.error("Existiert bereits.")
            zeige_tabelle("Lager", "Orte/Regale")

    elif auswahl == "Profil-Kuerzel":
        with st.form("form_profil", clear_on_submit=True):
            col1, col2 = st.columns(2)
            profil = col1.text_input("Profil (voller Name)")
            abk = col2.text_input("Abkuerzung (fuer Barcode)")
            _, col_btn = st.columns([3, 1])
            if col_btn.form_submit_button("Hinzufuegen", use_container_width=True) and profil and abk:
                try:
                    db.execute("INSERT INTO Profil_Abk (Profil, Abkuerzung) VALUES (?, ?)", (profil, abk.
                upper()))
                    st.success("Hinzugefuegt.")
                except db.IntegrityError:
                    st.error("Existiert bereits.")
            zeige_tabelle("Profil_Abk", "Profile")

    elif auswahl == "Material-Gruppen":
        formular_neue_materialgruppe()
        zeige_tabelle("Material-Gruppe", "Materialien")

    elif auswahl == "Einheiten":
        with st.form("form_einheit", clear_on_submit=True):
            symbol = st.text_input("Neues Einheit-Symbol (z.B. Stk, kg, m)")
            _, col_btn = st.columns([3, 1])
            if col_btn.form_submit_button("Hinzufuegen", use_container_width=True) and symbol:
                try:
                    db.execute("INSERT INTO Einheit (Symbol) VALUES (?)", (symbol,))
                    st.success("Hinzugefuegt.")
                except db.IntegrityError:
                    st.error("Existiert bereits.")
            zeige_tabelle("Einheit", "Einheiten")

    elif auswahl == "Typen":
        with st.form("form_typ", clear_on_submit=True):
            bez = st.text_input("Neuer Typ (z.B. Voll, Rest)")
            _, col_btn = st.columns([3, 1])
            if col_btn.form_submit_button("Hinzufuegen", use_container_width=True) and bez:
                try:
                    db.execute("INSERT INTO Typ (Bezeichnung) VALUES (?)", (bez,))
                    st.success("Hinzugefuegt.")
                except db.IntegrityError:
                    st.error("Existiert bereits.")
            zeige_tabelle("Typ", "Typen")

    elif auswahl == "Schwellenwerte":
```

```

aktuelle_grenze = logic.schrott_grenze_mm()
with st.form("form_schrott_grenze"):
    st.caption(
        "Eine gemeinsame Laengen-Grenze (mm) fuer zwei Stellen in der App:\n\n"
        "1. **Bei einer Materialentnahme**: liegt die berechnete Restlaenge darunter, "
        "fragt die App nach, ob das Stueck als Schrott entsorgt werden soll (statt es "
        "automatisch in die Reste-Liste zu legen).\n\n"
        "2. **Beim manuellen Hinzufuegen** neuer Ware: liegt die eingegebene Laenge darunter, "
        "wird das Stueck automatisch als Rest statt als voller Artikel gespeichert."
    )
    neue_grenze = st.number_input(
        "Schrott-/Rest-Grenze (mm)", min_value=0, step=50, value=int(aktuelle_grenze),
    )
    _, col_btn = st.columns([3, 1])
    if col_btn.form_submit_button("Speichern", type="primary", use_container_width=True):
        db.setze_konfiguration("SchrottGrenzeMM", int(neue_grenze))
        st.success(f"Grenze auf {int(neue_grenze)} mm gesetzt.")
        st.rerun()
st.caption(f"Aktuell aktiv: {int(aktuelle_grenze)} mm")

```

Modulkopf / Moduldaten

```

# -----
# Navigation
# -----

SEITEN = {
    "Dashboard": page_dashboard,
    "Artikel Liste": page_artikel_liste,
    "Reste": page_reste,
    "Verlauf": page_verlauf,
    "Papierkorb": page_papierkorb,
    "Artikel Gruppe": page_artikel_gruppe,
    "Einstellungen": page_einstellungen,
}

st.sidebar.title("Lager System")

seiten_namen = list(SEITEN.keys())
auswahl = st.sidebar.radio("Navigation", seiten_namen, key="nav_seite")

st.sidebar.divider()
st.sidebar.subheader("Schnellzugriff")
st.sidebar.caption(f"Automatische Voll/Rest-Zuordnung, Schwelle {logic.schrott_grenze_mm():.0f} mm")
if st.sidebar.button("■ Hinzufuegen", use_container_width=True, key="sb_hz"):
    dialog_hinzufuegen()
if st.sidebar.button(">■ Material entnehmen", use_container_width=True, key="sb_ent"):
    dialog_entnahme()
if st.sidebar.button("■ Bearbeiten / Loeschen", use_container_width=True, key="sb_be"):
    dialog_bearbeiten()
if st.sidebar.button("■■ Artikel Gruppe", use_container_width=True, key="sb_ag"):
    dialog_artikel_gruppe()
if st.sidebar.button("■ Material Gruppe", use_container_width=True, key="sb_mg"):
    dialog_material_gruppe()

SEITEN[auswahl]()

```

Migrations- und Betriebs-Skripte

Cloud-Migration, Start, Backup

migrate_sqlite_to_postgres.py

Einmaliges Script fuer den Cloud-Umzug: liest alle Tabellen aus der lokalen lager.db und spielt sie in eine Postgres-Datenbank ein.

Funktions-Uebersicht

Funktion / Bereich	Zweck
main()	Fuehrt die komplette Migration Tabelle fuer Tabelle aus.

Vollstaendiger Quellcode

Modulkopf / Moduldaten

"""Einmaliges Migrations-Script: kopiert alle Daten aus der lokalen SQLite-Datenbank (lager.db) in eine Postgres-Datenbank (z.B. Neon).

Nutzung:

```
export DATABASE_URL="postgresql://...neon.tech/..."
venv/bin/python migrate_sqlite_to_postgres.py
```

Kann gefahrlos mehrfach ausgefuehrt werden - leert vor jeder Tabelle die Zieltabelle, bevor die aktuellen Daten aus SQLite eingespielt werden.

"""

```
from __future__ import annotations
```

```
import os
import sqlite3
import sys
```

```
import psycpg2
```

```
TABELLEN = [
    "Lager", "Typ", "Einheit", "Konfiguration", "Profil_Abk", "Material_Gruppe",
    "Artikel_Gruppe", "Artikel_Liste", "Reste", "Papierkorb", "Verlauf",
]
```

```
# Tabellen mit SERIAL-Primaerschlussel - nach dem Kopieren muss die Postgres-
# interne Sequenz auf den hoechsten uebertragenen Wert gesetzt werden, sonst
# kollidiert der naechste automatisch vergebene Wert mit einer bestehenden ID.
```

```
SERIAL_PKS = {
    "Lager": "LagerID",
    "Typ": "TypeID",
    "Einheit": "EinheitID",
    "Artikel_Liste": "ID",
    "Reste": "ID",
    "Papierkorb": "ID",
    "Verlauf": "ID",
}
```

def main(...)

Fuehrt die komplette Migration Tabelle fuer Tabelle aus.

```
def main() -> None:
    database_url = os.environ.get("DATABASE_URL")
    if not database_url:
        print("Fehler: DATABASE_URL ist nicht gesetzt. Siehe Kommentar oben im Script.")
        sys.exit(1)
```

```
# Zielschema anlegen, falls noch nicht vorhanden (nutzt automatisch Postgres,
# da DATABASE_URL gesetzt ist).
```

```

import db as ziel_db
ziel_db.init_db()

quelle = sqlite3.connect("lager.db")
quelle.row_factory = sqlite3.Row

ziel_conn = psycopg2.connect(database_url)
try:
    cur = ziel_conn.cursor()
    for tabelle in TABELLEN:
        zeilen = quelle.execute(f"SELECT * FROM {tabelle}").fetchall()
        cur.execute(f"DELETE FROM {tabelle}")
        if not zeilen:
            print(f"{tabelle}: keine Zeilen in SQLite, Zieltabelle bleibt leer.")
            continue

        spalten = zeilen[0].keys()
        spalten_liste = ", ".join(spalten)
        platzhalter = ", ".join(["%s"] * len(spalten))
        for zeile in zeilen:
            werte = [zeile[s] for s in spalten]
            cur.execute(
                f"INSERT INTO {tabelle} ({spalten_liste}) VALUES ({platzhalter})", werte
            )

        if tabelle in SERIAL_PKS:
            pk = SERIAL_PKS[tabelle]
            # pg_get_serial_sequence() erwartet Tabellen-/Spaltennamen als
            # Text-Literal (keine SQL-Bezeichner) - Postgres speichert
            # unquotierte Namen intern klein, deshalb hier explizit .lower().
            cur.execute(
                f"SELECT setval(pg_get_serial_sequence('{tabelle.lower()}', '{pk.lower()}'), "
                f"COALESCE((SELECT MAX({pk}) FROM {tabelle}), 1))"
            )

        print(f"{tabelle}: {len(zeilen)} Zeilen uebertragen.")

    ziel_conn.commit()
finally:
    ziel_conn.close()
    quelle.close()

print("Fertig - alle Daten aus lager.db liegen jetzt auch in der Cloud-Datenbank.")

```

Modulkopf / Moduldaten

```

if __name__ == "__main__":
    main()

```

backup.sh

Taeglicher Backup-Job (Mac/Linux, per cron/launchd): kopiert lager.db mit Zeitstempel in einen Backup-Ordner.

```

#!/bin/bash
# Sicherheitskopie der Lagerdatenbank - wird bei jedem Start der App automatisch
# ausgefuehrt (siehe start.command / server.sh). Ein Backup pro Tag genuegt,
# deshalb ueberschreibt ein zweiter Start am selben Tag die Datei von heute.
set -euo pipefail
cd "$(dirname "$0")"

mkdir -p backups
datum=$(date +%Y-%m-%d)
rm -f "backups/lager_${datum}.db" "backups/lager_${datum}.db-wal" "backups/lager_${datum}.db-shm"
sqlite3 lager.db "VACUUM INTO 'backups/lager_${datum}.db'"

# Nur die letzten 30 Tage aufheben, aeltere Backups aufraeumen
ls -lt backups/lager_*.db 2>/dev/null | tail -n +31 | while IFS= read -r datei; do
    rm -f -- "$datei"
done

```

server.sh

Startet den Streamlit-Server im lokalen Netzwerk (Mac/Linux).

```

#!/bin/bash

```

```
# Wird vom Hintergrunddienst (LaunchAgent) beim Login automatisch gestartet
# und bei einem Absturz automatisch neu gestartet. Nicht per Doppelklick
# aufrufen - dafuer gibt es start.command.
set -euo pipefail
cd "$(dirname "$0")"

./backup.sh
source venv/bin/activate
exec streamlit run app.py --server.headless true
```

start.command

Doppelklick-Startpunkt fuer macOS - oeffnet ein Terminal und ruft server.sh auf.

```
cd "$(dirname "$0")"

if lsof -i :8501 -sTCP:LISTEN >/dev/null 2>&1; then
    echo "Die App laeuft bereits im Hintergrund - oeffne den Browser..."
    open "http://localhost:8501"
else
    ./backup.sh
    source venv/bin/activate
    streamlit run app.py
fi
```

windows-setup/backup.py

Windows-Pendant zu backup.sh (Python statt Shell, da Windows kein natives cron kennt).

```
"""Taegliche Sicherheitskopie der Lagerdatenbank - plattformunabhaengig (Mac/Windows).
Wird bei jedem Start der App automatisch ausgefuehrt (siehe start_server.bat / start.bat).
Ein Backup pro Tag genuegt, ein zweiter Start am selben Tag ueberschreibt die heutige Datei.
"""
import datetime as dt
import sqlite3
from pathlib import Path

BASIS = Path(__file__).resolve().parent
DB = BASIS / "lager.db"
BACKUPS = BASIS / "backups"

def sichern() -> None:
    BACKUPS.mkdir(exist_ok=True)
    datum = dt.date.today().isoformat()
    ziel = BACKUPS / f"lager_{datum}.db"
    for pfad in (ziel, ziel.with_suffix(".db-wal"), ziel.with_suffix(".db-shm")):
        pfad.unlink(missing_ok=True)

    conn = sqlite3.connect(DB)
    try:
        conn.execute(f"VACUUM INTO '{ziel.as_posix()}'")
    finally:
        conn.close()

    # Nur die letzten 30 Tage aufheben, aeltere Backups aufraeumen
    alle_backups = sorted(BACKUPS.glob("lager_*.db"), key=lambda p: p.stat().st_mtime, reverse=True)
    for alt in alle_backups[30:]:
        alt.unlink(missing_ok=True)

if __name__ == "__main__":
    sichern()
    print("Backup erstellt.")
```

windows-setup/start_server.bat

Windows-Pendant zu server.sh.

```
@echo off
REM Wird von der Windows-Aufgabenplanung automatisch gestartet (siehe Anleitung).
REM Nicht direkt per Doppelklick aufrufen - dafuer gibt es start.bat.
cd /d "%~dp0"
```

```
venv\Scripts\python.exe backup.py >> lagersystem.log 2>&1
venv\Scripts\streamlit.exe run app.py --server.headless true >> lagersystem.log 2>&1
```

windows-setup/start.bat

Doppelklick-Startpunkt fuer Windows.

```
@echo off
REM Manueller Fallback - per Doppelklick. Prueft ob der Hintergrunddienst schon
REM laeuft (dann nur Browser oeffnen), sonst wird die App direkt gestartet.
cd /d "%~dp0"

netstat -ano | findstr ":8501" | findstr "LISTENING" >nul
if %errorlevel%==0 (
    echo Die App laeuft bereits im Hintergrund - oeffne den Browser...
    start http://localhost:8501
) else (
    venv\Scripts\python.exe backup.py
    venv\Scripts\streamlit.exe run app.py
)
```

windows-setup/start_hidden.vbs

Startet den Server unsichtbar im Hintergrund (z.B. fuer Autostart), ohne sichtbares Konsolenfenster.

```
' Startet start_server.bat ohne sichtbares Konsolenfenster.
' Wird von der Windows-Aufgabenplanung aufgerufen (siehe ANLEITUNG_WINDOWS.md).
Dim fso, scriptDir, shell
Set fso = CreateObject("Scripting.FileSystemObject")
scriptDir = fso.GetParentFolderName(WScript.ScriptFullName)
Set shell = CreateObject("WScript.Shell")
shell.Run """" & scriptDir & "\start_server.bat""", 0, False
```